

第5章 回溯法

提纲

CONTENTS

5.1 回溯法概述

5.2 基于子集树框架的问题求解

5.3 基于排列树框架的问题求解

5.1 回溯法概述

5.1.1 问题的解空间

- 一个复杂问题的解决方案往往是由若干个小的决策（即选择）步骤组成的决策序列。
- 问题的解可以表示成解向量 $x = (x_0, x_1, \dots, x_{n-1})$ ，其中分量 x_i 对应第 i 步的选择， $x_i \in S_i$ ($0 \leq i \leq n-1$)。
- x 中各个分量 x_i 所有的取值的组合构成问题的解向量空间，简称为解空间，解空间一般用树形式来组织，也称为解空间树或者状态空间树。

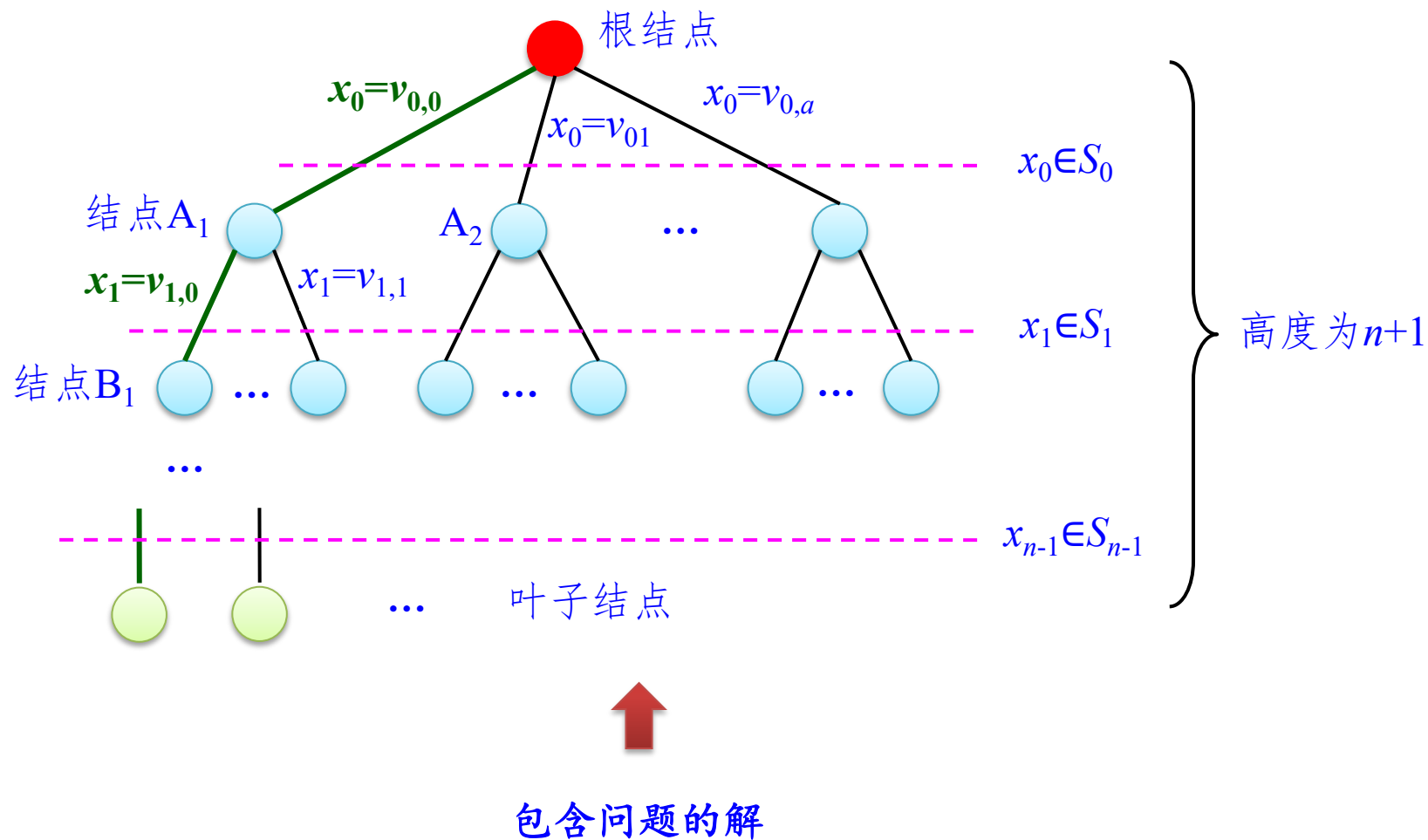
(1) 可能解由一个不等长向量组成，当物品 i ($1 \leq i \leq n$) 装入背包时，解向量中包含分量 i ，否则，解向量中不包含分量 i ，当 $n=3$ 时，其解空间是：

$\{(), (1), (2), (3), (1, 2), (1, 3), (2, 3), (1, 2, 3)\}$

(2) 可能解由一个等长向量 $\{x_1, x_2, \dots, x_n\}$ 组成，其中 $x_i=1$ ($1 \leq i \leq n$) 表示物品 i 装入背包， $x_i=0$ 表示物品 i 没有装入背包，当 $n=3$ 时，其解空间是：

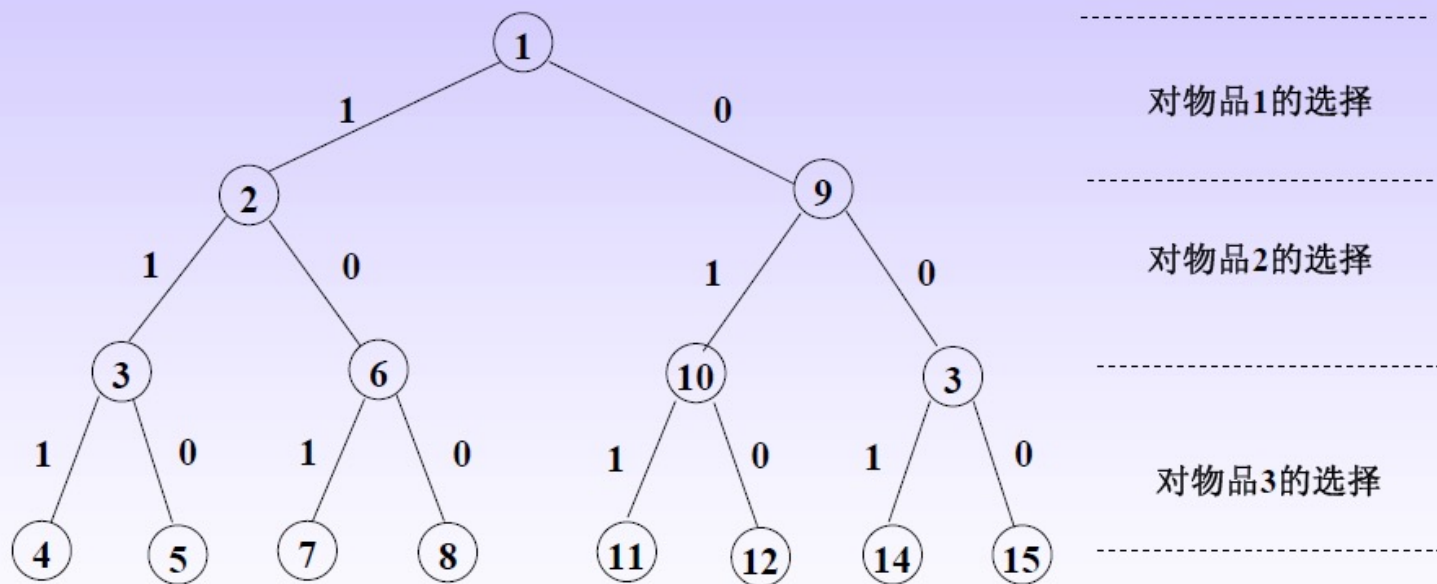
$\{(0, 0, 0), (0, 0, 1), (0, 1, 0), (1, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}$

解空间的一般结构



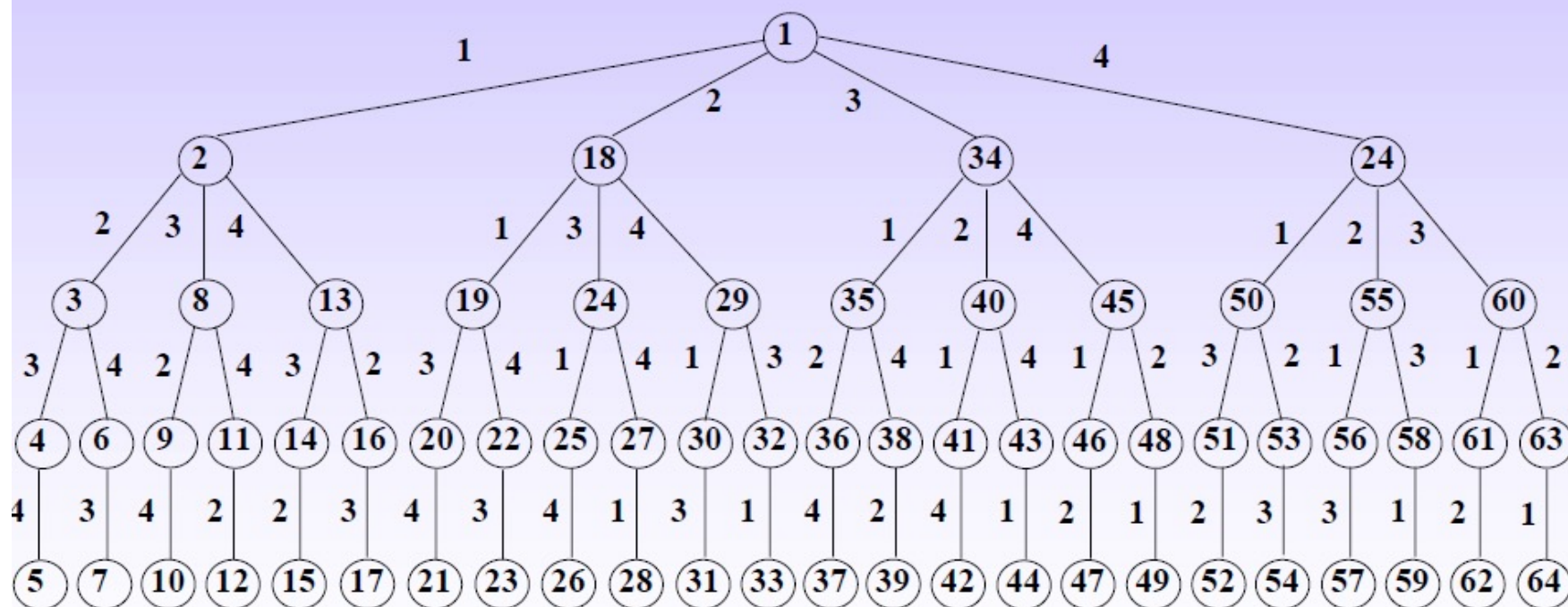
解空间的一般结构

对于 $n=3$ 的0/1背包问题，其解空间树如下图所示，树中的8个叶子结点分别代表该问题的8个可能解。



解空间的一般结构

对于 $n=4$ 的TSP问题，其解空间树如下图所示，树中的24个叶子结点分别代表该问题的24个可能解，例如结点5代表一个可能解，路径为 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ ，长度为各边代价之和。



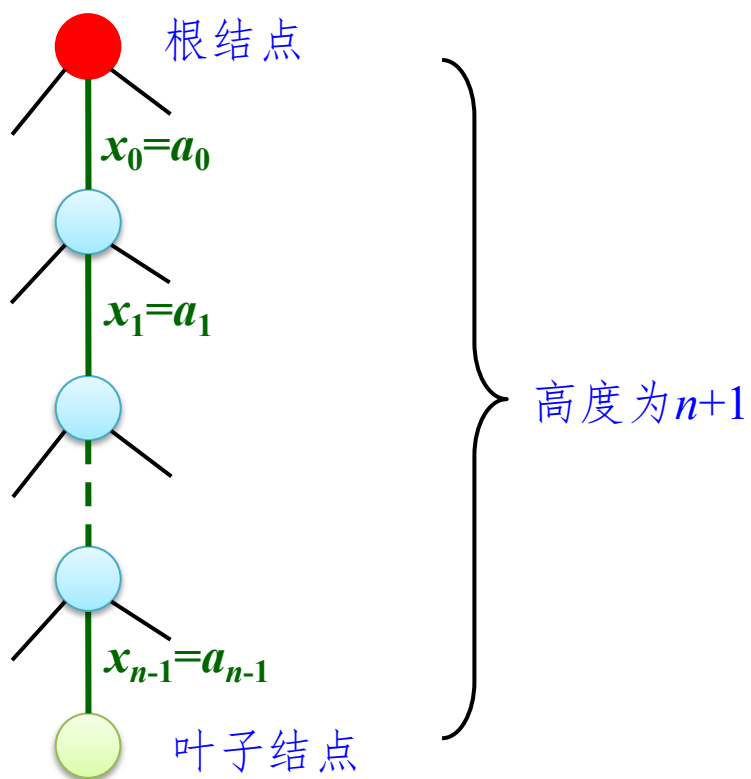
$n=4$ 的TSP问题的解空间树

5.1.2 什么是回溯法

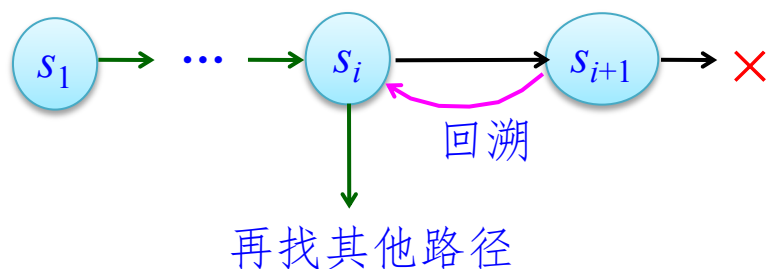
问题类型

- **求所有解类型**：给定一个约束函数，需要求所有满足约束条件的解。例如鸡兔同笼问题中，所有鸡兔头数为 a ，所有腿数为 b ，求所有头数为 a 、腿数为 b 的鸡兔数，设鸡兔数分别为 x 和 y ，则约束函数是 $x+y=a$ ， $2x+4y=b$ 。
- **求最优解类型**：除了约束条件外还包含目标函数，最后是求使目标函数最大或者最小的最优解。例如鸡兔同笼问题中，求所有鸡兔头数为 a 、所有腿数为 b 并且鸡最少的解，这就是一个求最优解问题，除了前面的约束函数外还包含目标函数 $\min(x)$ 。

回溯法是在解空间树中按照深度优先搜索方法从根结点出发搜索解。



回溯过程



几个概念

- **活结点**：指自身已生成但其孩子结点没有全部生成的结点。
- **扩展结点**：指正在产生孩子结点的结点。
- **死结点**：指其所有子结点均已产生的结点，此时应往回移动（回溯）至最近的一个活结点处。

回溯算法设计关键点

- 根据问题的特性确定结点是**如何扩展**的，不同的问题扩展方式是不同的。例如，在有向图中搜索从顶点s到顶点t的一条路径，其扩展十分简单，就是从一个顶点找所有相邻顶点。
- 在解空间中**按什么方式搜索解**，实际上树的遍历主要有先根遍历和层次遍历，前者就是**深度优先搜索（DFS）**，后者就是**广度优先搜索（BFS）**。
- **回溯法**就是采用深度优先搜索解，**分支限界法**则是采用广度优先搜索解。
- 解空间通常十分庞大，如何高效地找到问题的解，通常采用一些**剪支**的方法实现。



回溯法 = DFS + 剪支

剪支：在解空间中搜索时提早终止某些分支的无效搜索，减少搜索的结点个数但不影响最终结果，从而提高了算法的时间性能。

剪支策略

- **可行性剪支：**在扩展结点处剪去不满足约束条件的分支。
- **最优性剪支：**用限界函数剪去得不到最优解的分支。
- **交换搜索顺序：**在搜索中改变搜索的顺序，比如原先是递减顺序，可以改为递增顺序，或者原先是无序，可以改为有序，这样可能减少搜索的总结点。

回溯法求解的一般步骤

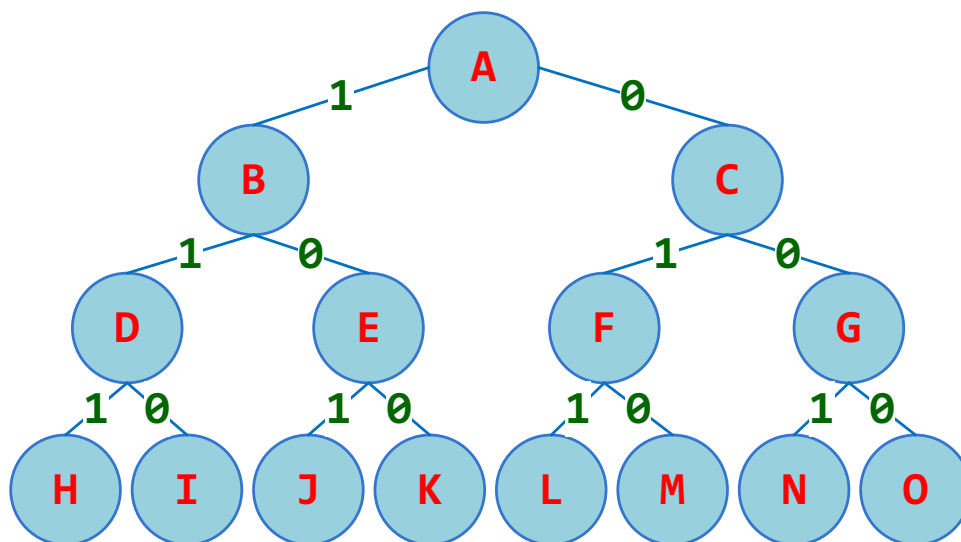
- ① 针对给定的问题确定其解空间，其中至少包含问题的一个解或者最优解。
- ② 确定结点的扩展规则。
- ③ 采用深度优先搜索方法搜索解空间，并在搜索过程中尽可能采用剪支函数避免无效搜索。

【0-1背包问题】 $n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45,25,25\}$

开始时, $Cr=C=30$, $V=0$

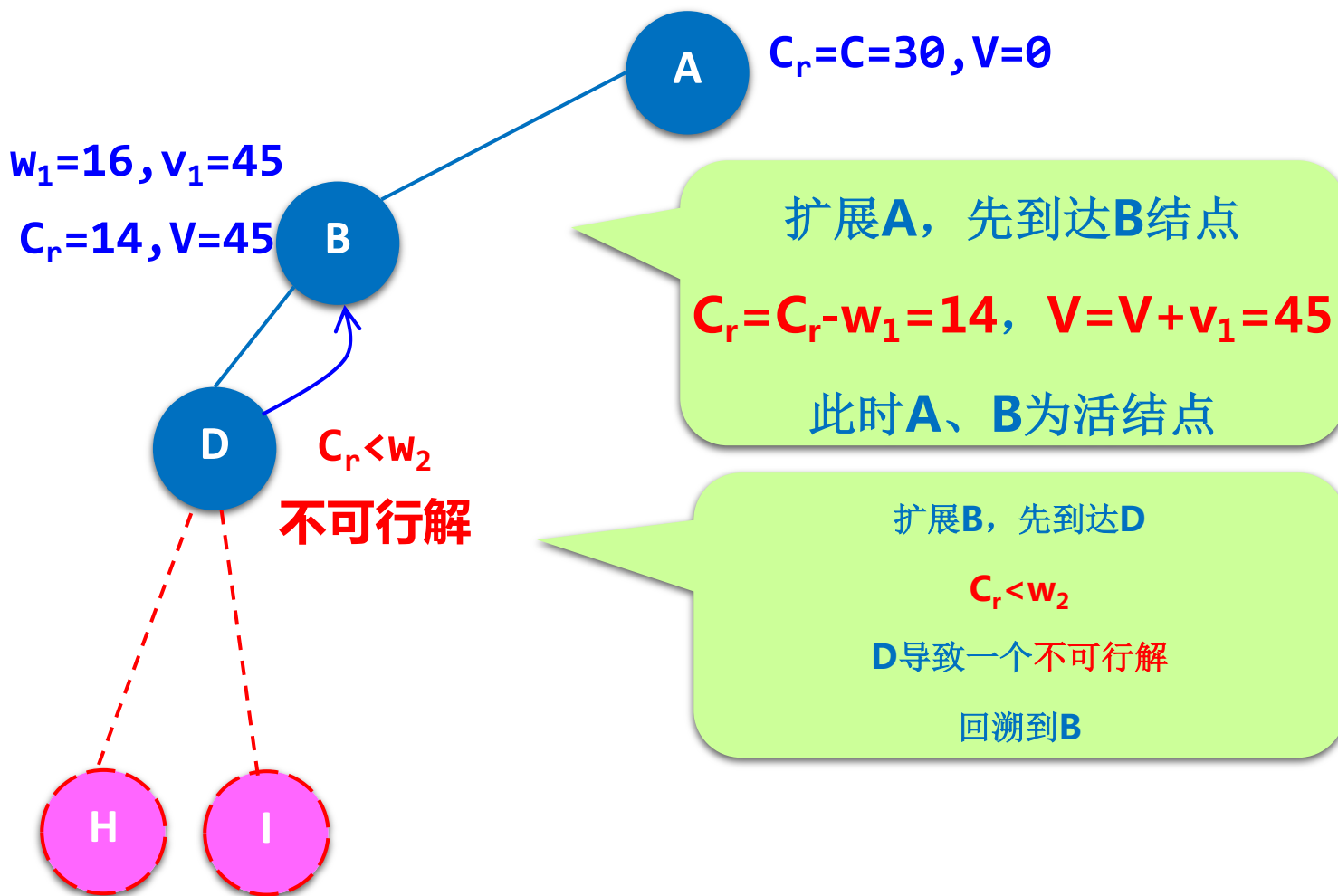
C 为容量, Cr 为剩余空间, V 为价值。

A 为唯一活结点, 也是当前扩展结点。。



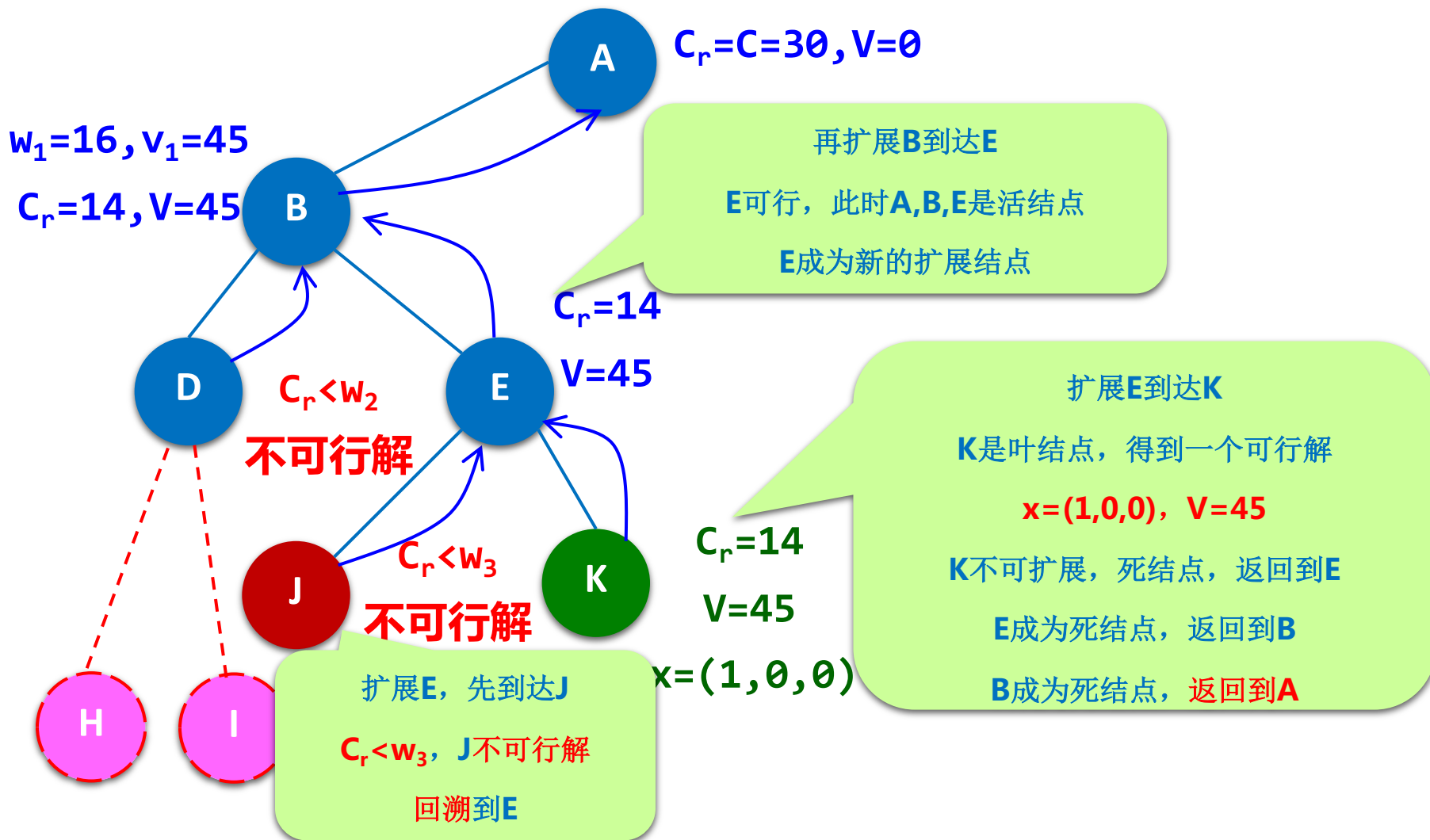
解

• $n=3$, $C=30$, $w=\{16,15,15\}$, $v=\{45,25,25\}$



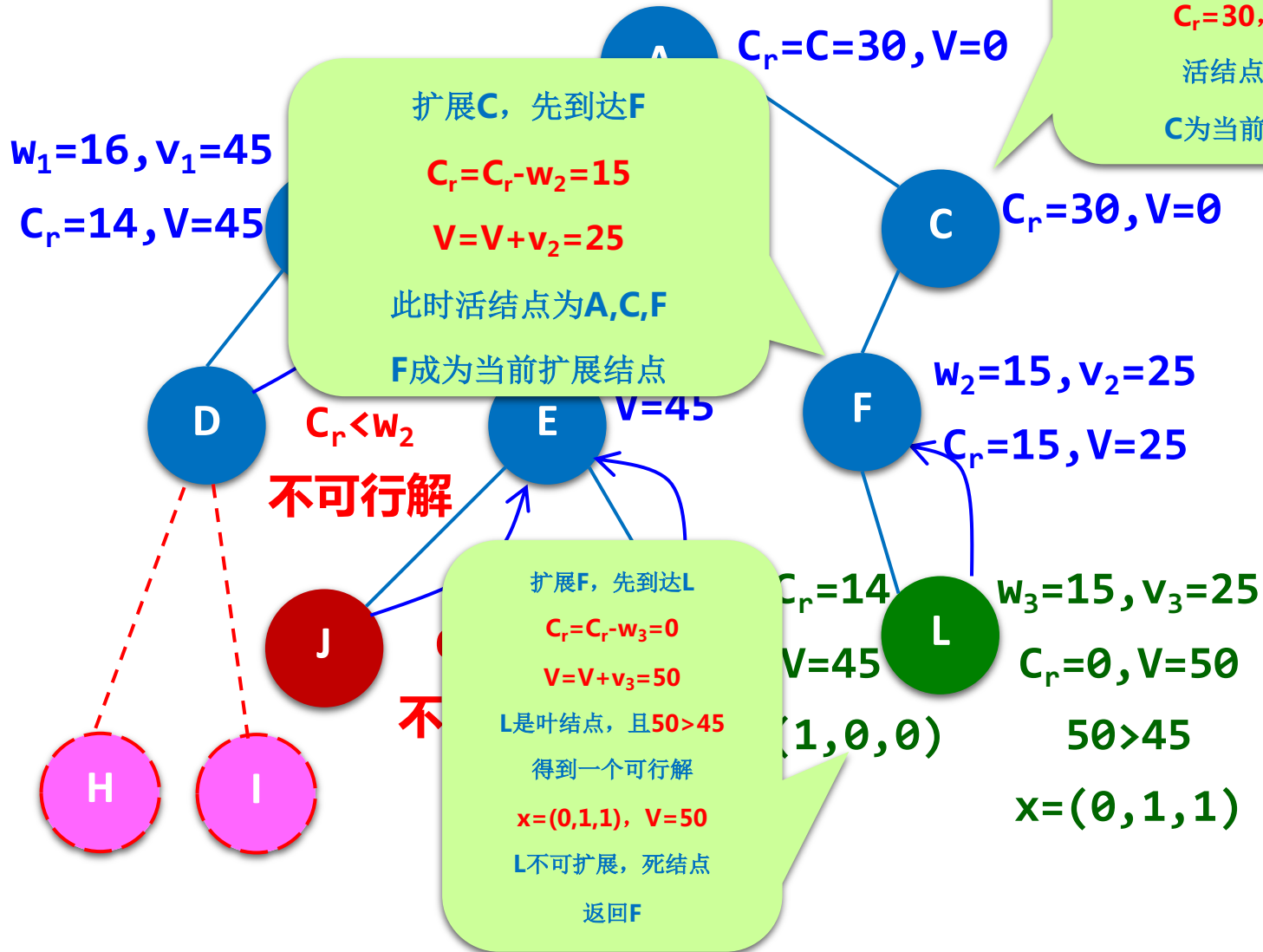
解

• $n=3$, $C=30$, $w=\{16,15,15\}$, $v=\{45,25,25\}$



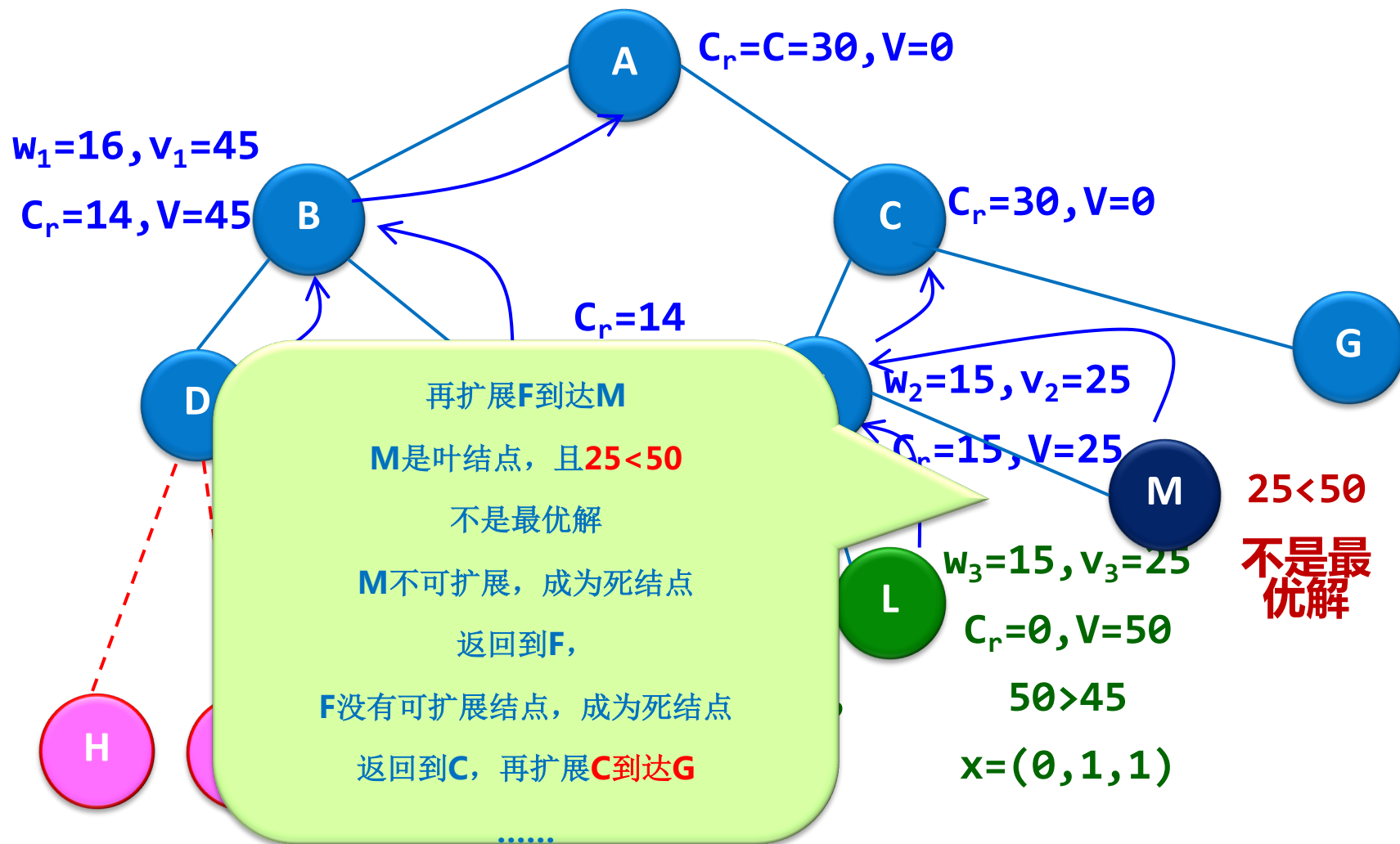
解

$n=3$, $C=30$, $w=\{16,15,15\}$, $v=\{45,25,25\}$



解

• $n=3$, $C=30$, $w=\{16,15,15\}$, $v=\{45,25,25\}$

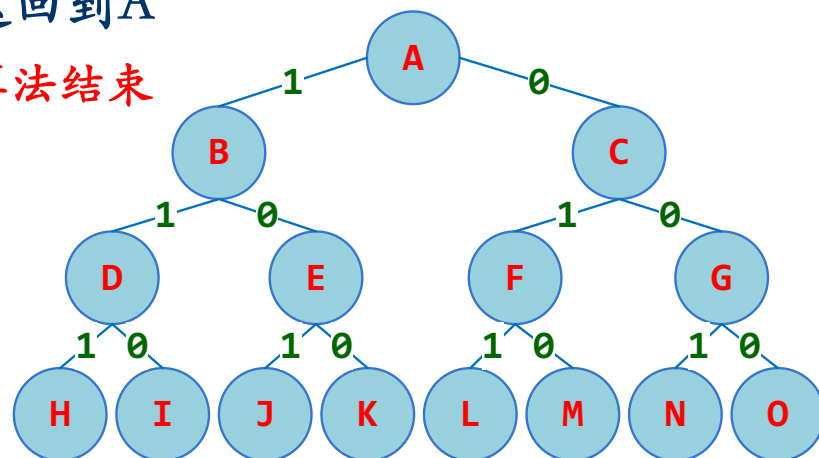


解

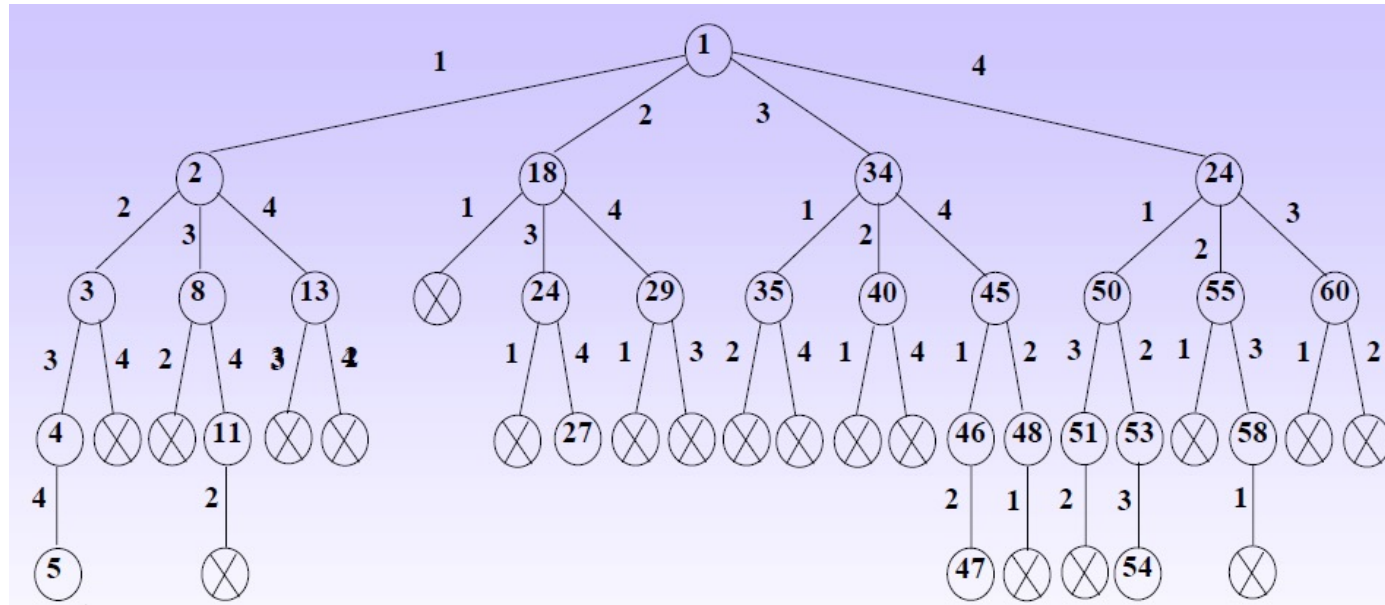
• $n=3$, $C=30$, $w=\{16,15,15\}$, $v=\{45,25,25\}$

■ 扩展到G以后……

- $C_r=30$, $V=0$,
活结点为A、C、G, G为当前扩展结点
- 扩展G, 先到达N, N是叶结点, 且 $25 < 50$, 不是最优解, 又N不可扩展, 返回到G
- 再扩展G到达O, O是叶结点, 且 $0 < 50$, 不是最优解, 又O不可扩展, 返回到G
- G没有可扩展结点, 成为死结点, 返回到C
- C没有可扩展结点, 成为死结点, 返回到A
- A没有可扩展结点, 成为死结点, 算法结束
- 最优解 $X=(0,1,1)$, 最优值50。



【TSP问题】 某个售货员要到若干城市去推销商品，已知各城市之间的路程（或旅费）。他要选定一条从驻地城市出发，经过每个城市一遍，最后回到驻地的路线，使总的路程（或总旅费）最小。



再如，对于 $n=4$ 的TSP问题，其代价矩阵如下图所示，

$$C = \begin{pmatrix} \infty & 3 & 6 & 7 \\ 12 & \infty & 2 & 8 \\ 8 & 6 & \infty & 2 \\ 3 & 7 & 6 & \infty \end{pmatrix}$$

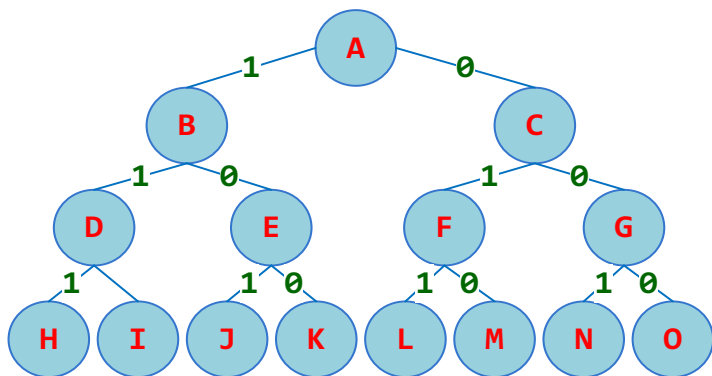
TSP问题的代价矩阵

5.1.3 回溯法框架

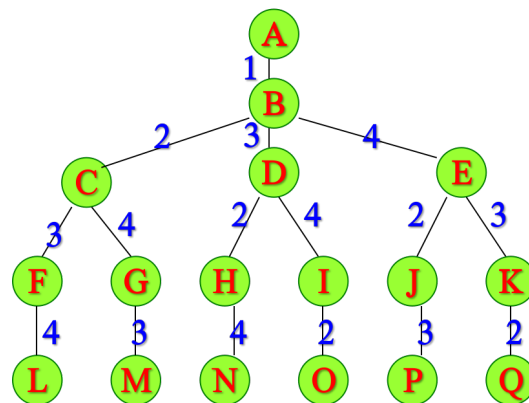
解空间类型

- **子集树**：所给的问题是从 n 个元素的集合 S 中找出满足某种性质的子集。例如在整数数组 a 中求和为目标值 $target$ 的所有解，每个元素 $a[i]$ 只有选择和不选择两种方式。
- **排列树**：所给的问题是确定 n 个元素满足某种性质的排列。例如求全排列问题的解空间就是典型的排列树。

子集树



排列树



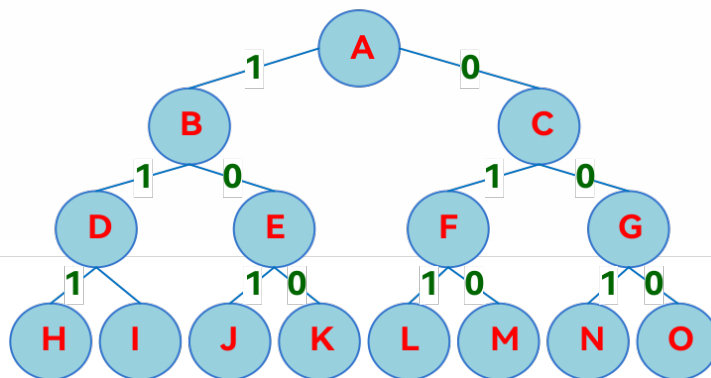
1. 解空间为子集树

```
int x[n];  
void backtrack(int i)  
{  if(i>n)  
    输出一个解;  
    else  
    {  for (j=下界;j<=上界;j++)  
        {  x[i]=j;  
            ...  
            if (constraint(i,j) && bound(i,j))  
                backtrack(i+1);  
            回溯x[i];  
            ...  
        }  
    }  
}
```

//x存放解向量, 这里作为全局变量
//求解子集树的递归框架
//搜索到叶子结点, 输出一个可行解

//用j表示x[i]的所有可能候选值
//产生一个可能的解分量
//其他操作
//剪支
//满足条件, 继续下一层

子集树



0/1背包问题

- 有 n 个编号为 $0 \sim n-1$ 的物品，重量为 $w=\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $v=\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。
- 从这些物品中选取全部或者部分物品装入该背包中，每个物品要么选中要么不选中，即物品不能被分割，找到选中物品不仅能够放到背包中而且**价值最大**的方案（**最大价值**）。

物品编号	重量 w	价值 v
0	5	4
1	3	4
2	2	3
3	1	1

$W=6$
结果为8

解 1) 存储结构设计

```
struct Goods                                //物品结构体类型
{
    int no;                                //物品的编号
    int w;                                  //物品的重量
    int v;                                  //物品的价值
    Goods(int no1,int w1,int v1)           //构造函数
    {
        no=no1;
        w=w1;
        v=v1;
    }
    bool operator<(const Goods&s) const    //用于按v/w递减排序
    {
        return (double)v/w>(double)s.v/s.w;
    }
};
```

解向量 $x = (x_0, x_1, \dots, x_{n-1})$ ， $x_i=1$ 表示选择物品 i ， $x_i=0$ 表示不选择物品 i ，最优解向量用 bestx 表示，最大价值用 bestv 表示（初始为0）。

2) 左剪支

- 左剪支使用约束函数/条件（不超出背包容量）。
- 当搜索到第 i ($0 \leq i < n$) 层的某个结点时, cw 表示当前选择的物品重量和（其中不包含 $w[i]$ ）。
- 检查当前物品被选中后总重量是否超过 W , 若超过则剪支, 即仅仅扩展满足 $cw + w[i] \leq W$ 的左孩子结点。

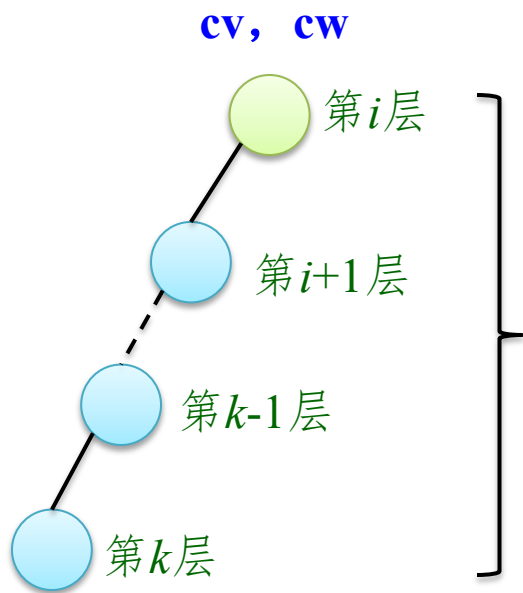
3) 右剪支

- 采用限界函数（上界函数）。
- 将所有物品 g 按单位重量价值递减排序。
- 例如前面表排序后的结果如下，序号 i 发生了改变，后面改为按 i 而不是物品编号 no 的顺序搜索。

序号 i	物品编号 no	重量 w	价值 v	v/w
0	2	2	3	1.5
1	1	3	4	1.3
2	3	1	1	1
3	0	5	4	0.8

第*i*层结点： **cw**表示当前选择的物品重量和（不含 $w[i]$ ），**cv**表示当前选择的物品价值和（不含 $v[i]$ ）。

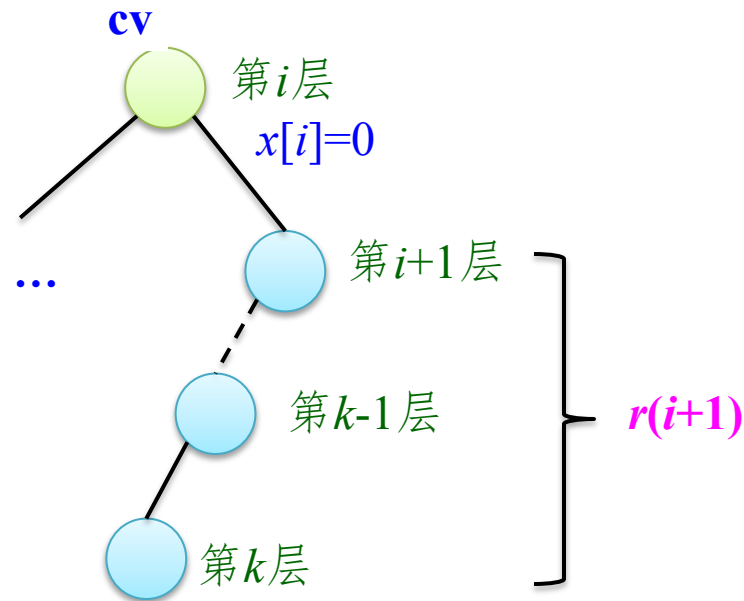
此时背包剩余容量为 $rw=W-cw$ ，如果按背包容量连续选择物品*i*及其后面的物品，直到装满，其价值一定是最大的（因为物品按 v/w 递减排序），**最大价值为 $r(i)$** ：



试探： 物品*i*~*k-1*可以整个装入，物品*k*只能部分装入

$$r(i) = \sum_{j=i}^{k-1} v_j + \left(rw - \sum_{j=i}^{k-1} w_j \right) (v_k / w_k)$$

第*i*层结点：如果不选择物品*i*时，此时背包剩余容量为 $rw=W-cw$ 。余下重量的物品的最大价值 $r(i+1)$ ：



$$r(i) = \sum_{j=i}^{k-1} v_j + \left(rw - \sum_{j=i}^{k-1} w_j \right) (v_k / w_k)$$

限界函数

不选择物品 i 时这条路径走下去能够选择物品的最大价值为 $\text{bound}(\text{cw}, \text{cv}, i+1)$ 。

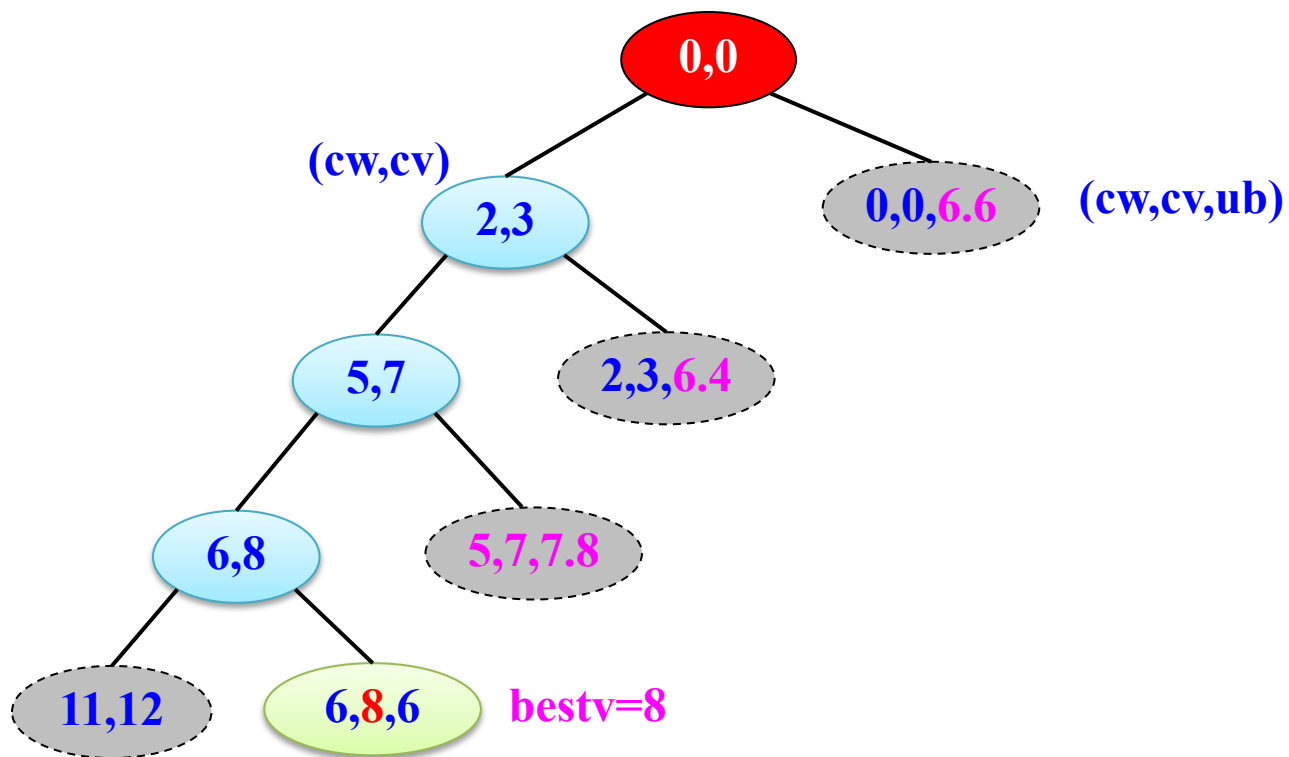
对于第 i 层结点实参为 $i+1$

```
double bound(int cw, int cv, int i)    //计算第i层结点的上界函数值
{   int rw = W - cw;                  //背包的剩余容量
    double b = cv;                    //表示物品价值的上界值
    int j = i;
    while (j < n && g[j].w <= rw)
    {   rw -= g[j].w;                  //选择物品j
        b += g[j].v;                  //累计价值
        j++;
    }
    if (j < n)                          //最后一个物品只能部分装入
        b += (double)g[j].v / g[j].w * rw;
    return b;
}
```

■ 剪支：仅仅扩展 $\text{bound}(\text{cw}, \text{cv}, i+1) > \text{bestv}$ 的右孩子结点。

右剪支实例：W=6

序号i	物品编号no	重量w	价值v	v/w
0	2	2	3	1.5
1	1	3	4	1.3
2	3	1	1	1
3	0	5	4	0.8



void dfs (int cw,int cv,vector<int>&x,int i)	//回溯算法
{ if (i>=n)	//找到一个叶子结点
{ if (cw<=W && cv>bestv)	//找到一个满足条件的更优解,保存它
{ bestv=cv;	
bestx=x;	
}	
}	
else	//没有到达叶子结点
{ if(cw+g[i].w<=W)	//左剪支
{ x[i]=1;	//选取物品i
dfs(cw+g[i].w,cv+g[i].v,x,i+1);	
}	
double b=bound(cw,cv,i+1);	//计算上界时从物品i+1开始
if(b>bestv)	//右剪支
{ x[i]=0;	//不选取物品i
dfs(cw,cv,x,i+1);	
}	
}	
}	

【算法分析】最坏情况下算法的时间复杂度为 $O(2^n)$ 。

2. 解空间为排列树

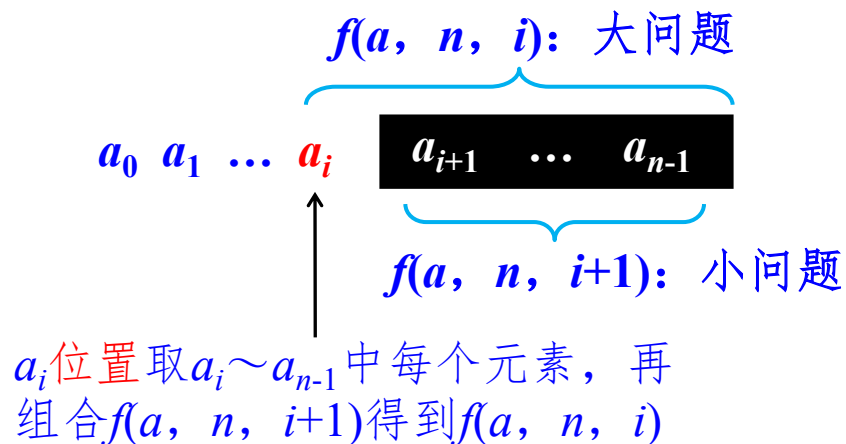
【排列数】有一个含 n 个整数的数组 a ，所有元素均不相同，求其所有元素的全排列。

例如， $a=\{1, 2, 3\}$ ，得到结果是 $(1, 2, 3)$ 、 $(1, 3, 2)$ 、 $(2, 3, 1)$ 、 $(2, 1, 3)$ 、 $(3, 1, 2)$ 、 $(3, 2, 1)$ 。

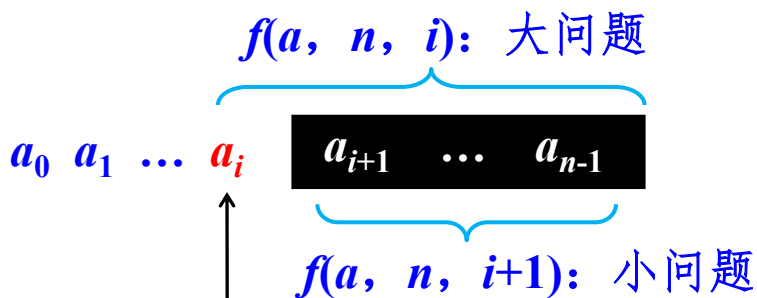
解

用数组 a 存放初始数组 a 的一个排列，采用递归法求解。

设 $f(a, n, i)$ 表示求 $a[i..n-1]$ （共 $n-i$ 个元素）的全排列，为大问题， $f(a, n, i+1)$ 表示求 $a[i+1..n-1]$ （共 $n-i-1$ 个元素）的全排列，为小问题。



- a_i : a_i 与 a_i 交换, $f(a, n, i+1) \Rightarrow$ 以 a_i 开头的 $a[i..n-1]$ 的全排列
- a_{i+1} : a_i 与 a_{i+1} 交换, $f(a, n, i+1) \Rightarrow$ 以 a_{i+1} 开头的 $a[i..n-1]$ 的全排列
- ...
- a_{n-1} : a_i 与 a_{n-1} 交换, $f(a, n, i+1) \Rightarrow$ 以 a_{n-1} 开头的 $a[i..n-1]$ 的全排列



a_i 位置取 $a_i \sim a_{n-1}$ 中每个元素，再组合 $f(a, n, i+1)$ 得到 $f(a, n, i)$

求 a 的全排列的递归模型 $f(a, n, i)$

$f(a, n, i) \equiv$ 输出产生的解

$f(a, n, i) \equiv$ 对于 $j=i \sim n-1$:

$a[i]$ 与 $a[j]$ 交换位置;

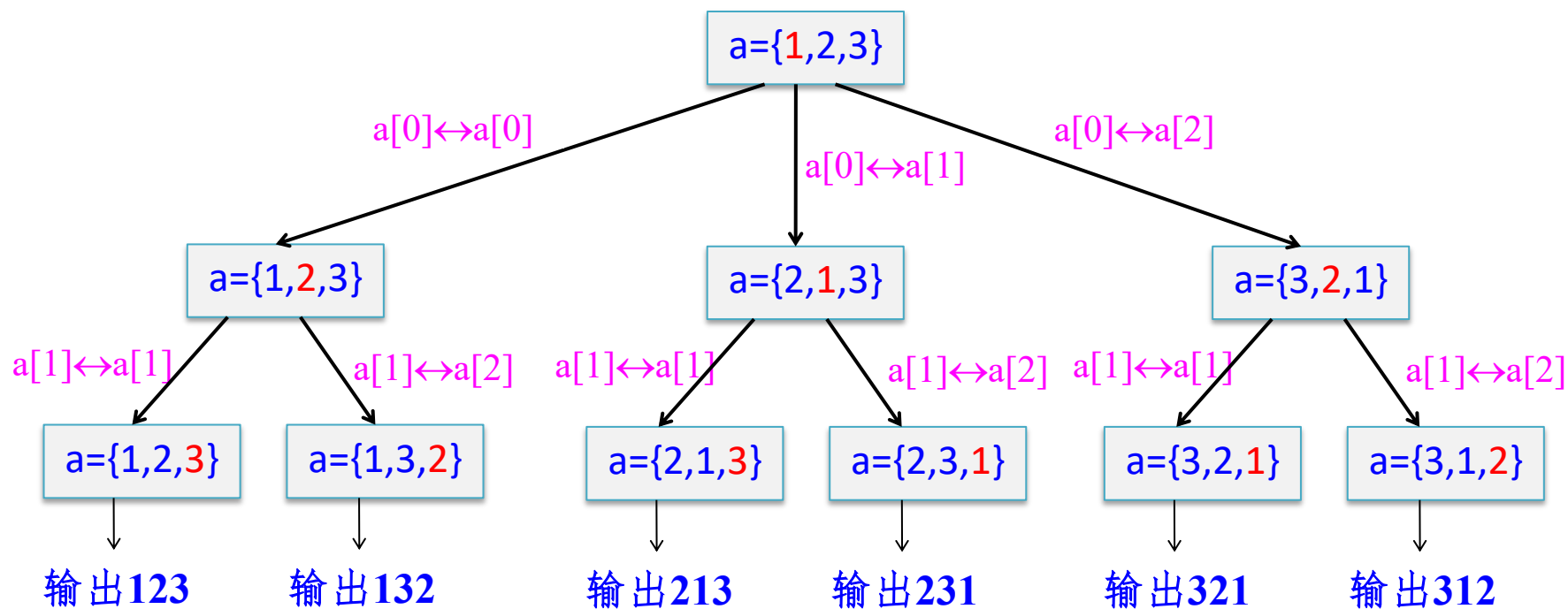
$f(a, n, i+1)$;

将 $a[i]$ 与 $a[j]$ 交换位置 (回溯)

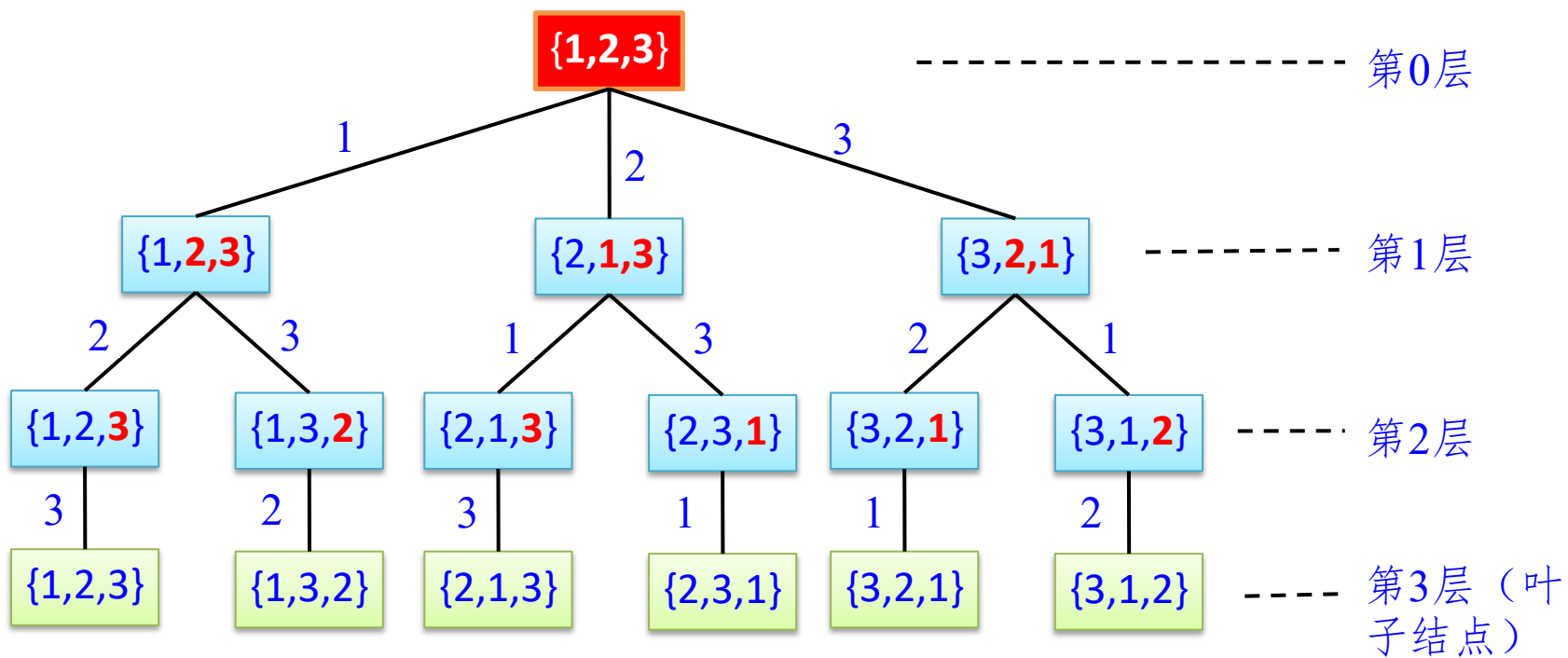
当 $i=n-1$ 时

其他

求 $a=\{1, 2, 3\}$ 全排列



标准解空间



```
void dfs(vector<int>& a,int i)
```

```
{ int n=a.size();
```

```
  if (i>=n-1)
```

```
    disp(a);
```

```
  else
```

```
  { for (int j=i;j<n;j++)
```

```
    { swap(a[i],a[j]);
```

```
      dfs(a,i+1);
```

```
      swap(a[i],a[j]);
```

```
    }
```

```
  }
```

```
}
```

```
void perm(vector<int>& a)
```

```
{
```

```
  dfs(a,0);
```

```
}
```

//递归算法

//递归出口

//交换a[i]与a[j]

//交换a[i]与a[j]: 恢复

//求a的全排列

排列树算法基本框架

```
int x[n];  
void backtrack(int i)  
{  if(i>n)  
    输出结果;  
    else  
    {  for (j=i;j<=n;j++)  
        {  ...  
            swap(x[i],x[j]);  
            if (constraint(i,j) && bound(i,j))  
                backtrack(i+1);  
            swap(x[i],x[j]);  
            ...  
        }  
    }  
}
```

//x存放解向量，并初始化
//求解排列树的递归框架
//搜索到叶子结点,输出一个可行解

//用j枚举x[i]所有可能候选值
//第i层的结点选择x[j]的操作
//通过交换来实现
//满足约束条件和限界函数，进入下一层
//恢复状态:回溯
//第i层的结点选择x[j]的恢复操作

5.1.4 回溯法算法时间分析

- 解空间树共有 $n+1$ 层（根结点为第0层，叶子结点为第 n 层）。
- 第1层有 m_0 个结点，每个结点有 m_1 个子结点。
- 第2层有 m_0m_1 个结点，同理，第3层有 $m_0m_1m_2$ 个结点，依此类推，第 n 层有 $m_0m_1\dots m_{n-1}$ 个结点。
- 则采用回溯法求所有解的算法的执行时间为

$$T(n)=m_0+m_0m_1+m_0m_1m_2+\dots+m_0m_1m_2\dots m_{n-1}。$$

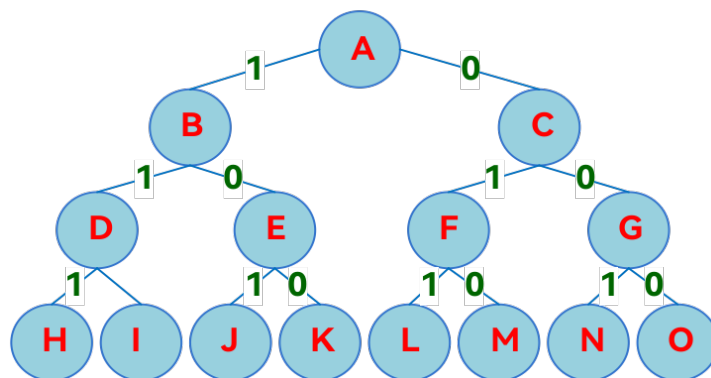
- 例如，在子集树中有 $m_0=m_1=\dots=m_{n-1}=c$ ，对应算法的时间复杂度为 $O(c^n)$ ，在排列树中有 $m_0=n$ ， $m_1=n-1$ ， $\dots$ ， $m_{n-1}=1$ ，对应算法的时间复杂度为 $O(n!)$ 。

5.2 基于子集树框架的问题求解

5.2.1 简单装载问题

- 有 n 个集装箱要装上一艘载重量为 t 的轮船，其中集装箱 i ($0 \leq i \leq n-1$) 的重量为 w_i 。不考虑集装箱的体积限制，现要选出重量和小于等于 t 并且尽可能重的若干集装箱装上轮船。
- 例如， $n=5$ ， $t=10$ ， $w=\{5, 2, 6, 4, 3\}$ 时，其最佳装载方案有两种即 $(1, 1, 0, 0, 1)$ 和 $(0, 0, 1, 1, 0)$ ，对应集装箱重量和达到最大值 t 。

子集树



解

```
int n=5,t=10;  
int w[]={5,2,6,4,3};  
vector<int> bestx;  
int bestw=0;
```

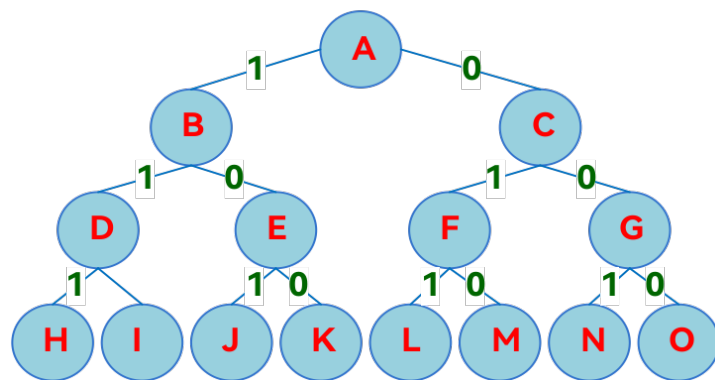
//一个测试实例

//各集装箱重量,不用下标0的元素

//存放最优解向量

//存放最优解的总重量,初始化为0

子集树



```
int n=5,t=10;
```

//一个测试实例

```
int w[]={5,2,6,4,3};
```

//各集装箱重量,不用下标0的元素

```
vector<int> bestx;
```

//存放最优解向量

```
int bestw=0;
```

//存放最优解的总重量,初始化为0

```
void dfs(int cw,int rw,vector<int>&x,int i)
```

```
{ if (i>=n)
```

//递归算法

```
{ if (cw>bestw)
```

//找到一个叶子结点

```
{ bestw=cw;
```

//找到一个满足条件的更优解

```
bestx=x;
```

//保存更优解

```
}
```

```
}
```

```
else
```

```
{ rw-=w[i];
```

//没有到达叶子结点

```
if (cw+w[i]<=t)
```

//求剩余集装箱的重量和

```
{ x[i]=1;
```

//左孩子结点剪支

```
cw+=w[i];
```

//选取集装箱i

```
dfs(cw,rw,x,i+1);
```

//累计当前所选集装箱的重量和

```
cw-=w[i];
```

//恢复所选集装箱的重量和(回溯)

```
}
```

```
if (cw+rw>bestw)
```

//右孩子结点剪支

```
{ x[i]=0;
```

//不选择集装箱i

```
dfs(cw,rw,x,i+1);
```

```
}
```

```
rw+=w[i];
```

//恢复剩余集装箱的重量和(回溯)

```
}
```

```
}
```

```
void loading()                //求解简单装载问题
{
    bestx.resize(n);
    vector<int> x(n);
    int rw=0;
    for (int i=0;i<n;i++)
        rw+=w[i];
    dfs(0,rw,x,0);
}
```

该算法的解空间树中有 $2^{n+1}-1$ 个结点，所以最坏情况下算法的时间复杂度为 $O(2^n)$ 。

5.2.2 n皇后问题

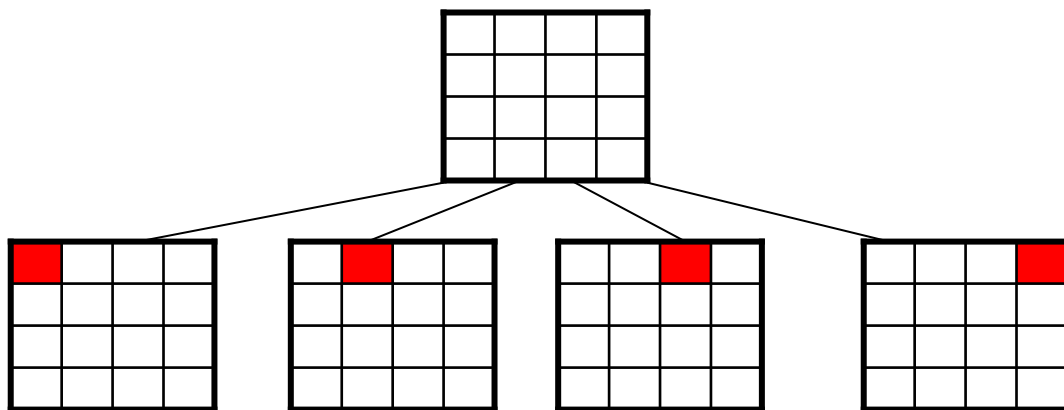
八皇后问题是十九世纪著名的数学家高斯于1850年提出的。

问题是：在 8×8 的棋盘上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上。可以把八皇后问题扩展到n皇后问题，即在 $n \times n$ 的棋盘上摆放n个皇后，使任意两个皇后都不能处于同一行、同一列或同一斜线上。

- 在 $n \times n$ ($n \geq 4$) 的方格棋盘上放置n个皇后，并且每个皇后不同行、不同列、不同左右对角线（否则称为有冲突）。
- 要求给出n个皇后全部解。

解

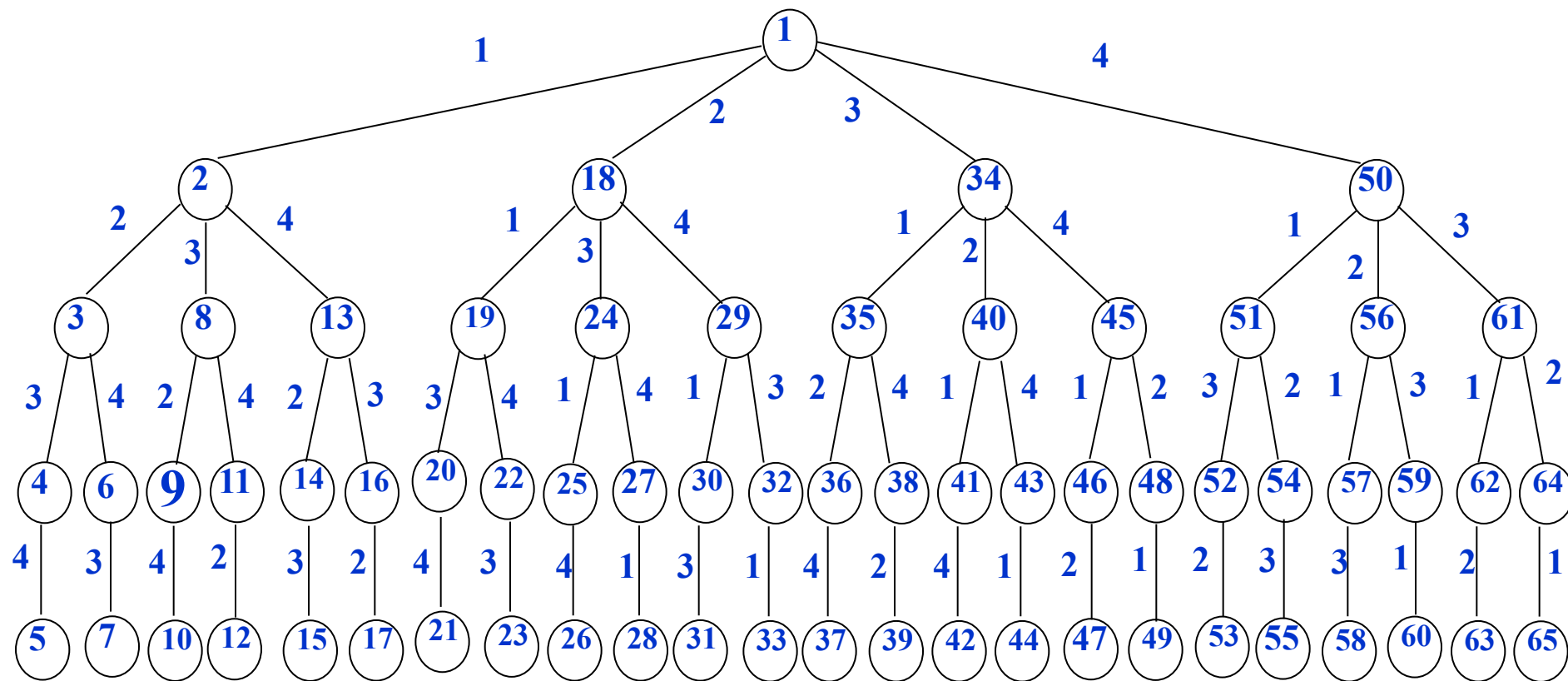
- 确定问题状态：问题的状态即棋盘的布局状态。
- 构造状态空间树：状态空间树的根为空棋盘，每个布局的下一步可能布局是该布局结点的子结点。
 - 由于可以预知，在每行中有且只有一个皇后，因此可采用逐行布局的方式，即每个布局有 n 个子结点。



- 设4个皇后为 x_i ，分别在第 i 行($i=1, 2, 3, 4$);
- 问题的解状态：可以用 $(1, x_1), (2, x_2), \dots, (4, x_4)$ 表示4个皇后的位置;
- 由于行号固定，可简单记为： (x_1, x_2, x_3, x_4) ; 例如： $(4, 2, 1, 3)$
- 问题的解空间： $(x_1, x_2, x_3, x_4), 1 \leq x_i \leq 4 (i=1, 2, 3, 4)$, 共 $4!$ 个状态;

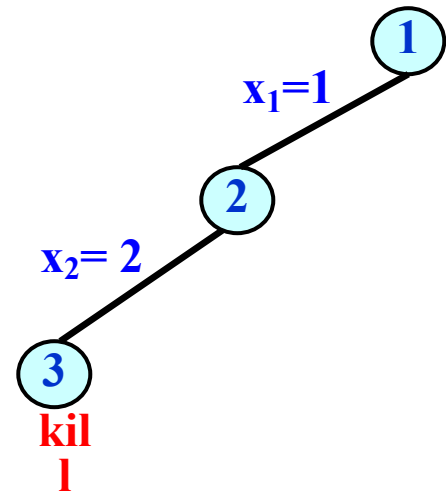
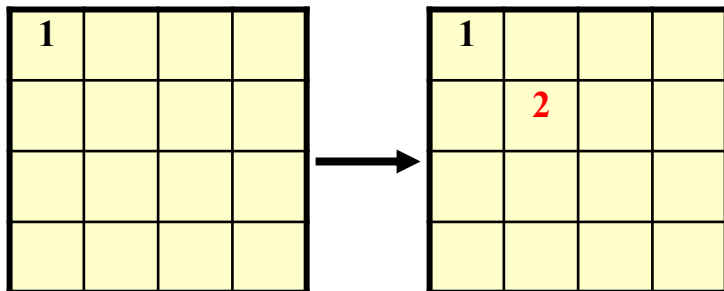
解

4皇后问题解空间的树结构



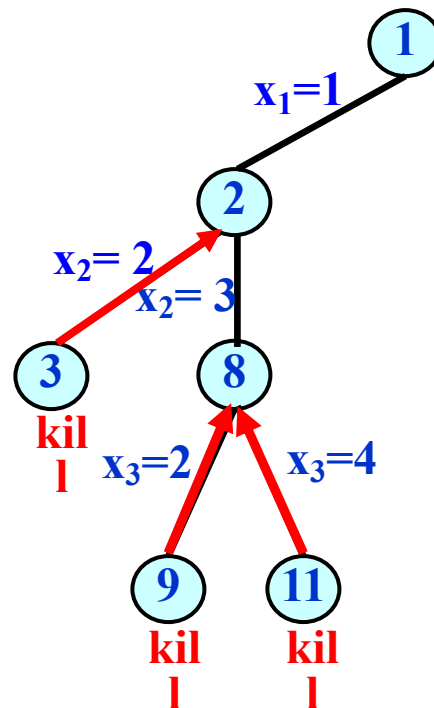
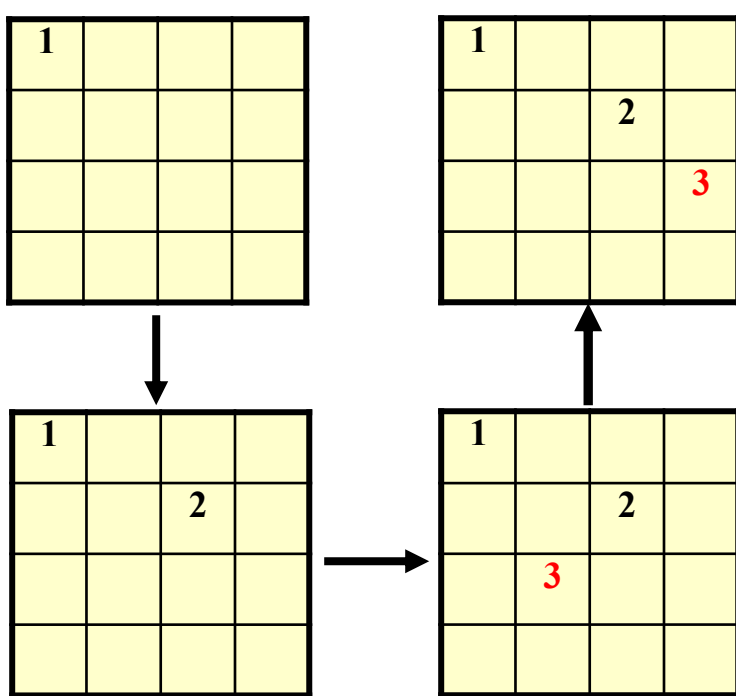
4皇后问题

- 开始把根结点1作为唯一的活结点, 根结点就成为E-结点而且路径为(); 接着生成子结点, 假设按自然数递增的次序来生成子结点, 那么结点2被生成, 这条路径为(1), 即把皇后1放在第1列上。
- 结点2变成E-结点, 它再生成结点3, 路径变为(1, 2), 即皇后1在第1列上, 皇后2在第2列上, 所以结点3被杀死, 此时应回溯。



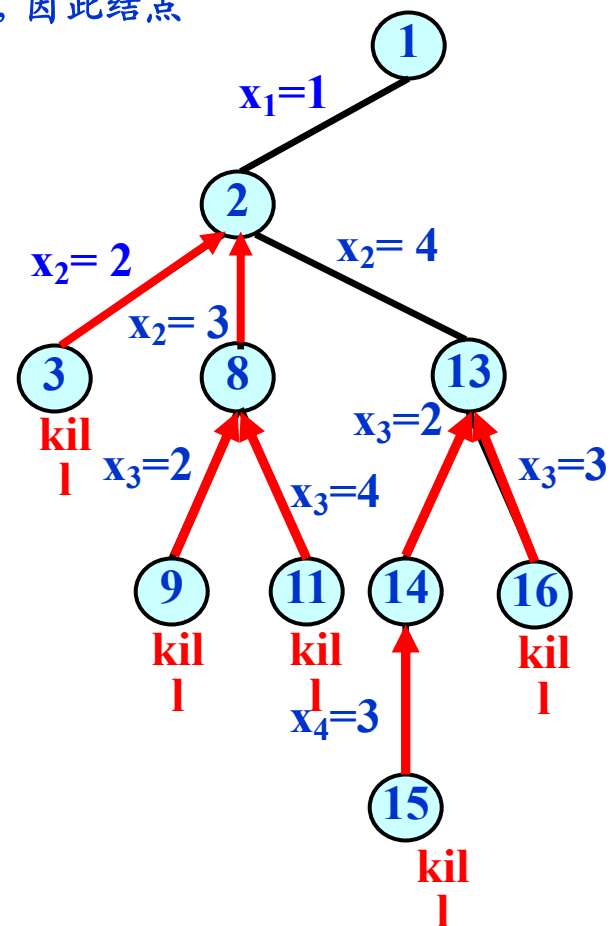
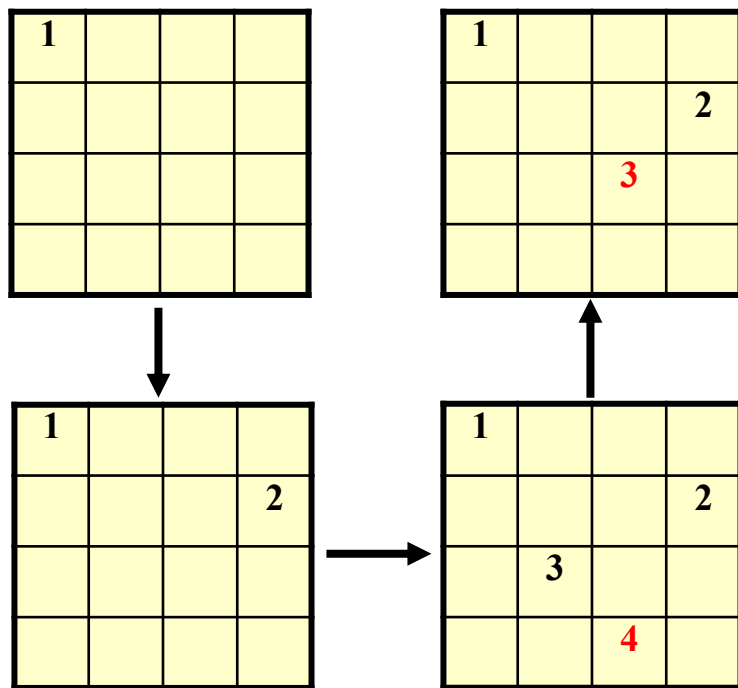
4皇后问题

- 回溯到结点2生成结点8, 路径变为(1, 3), 则结点8成为E-结点, 它生成结点9和结点11都会被杀死(即它的儿子表示不可能导致答案的棋盘格局), 所以结点8也被杀死, 应回溯。



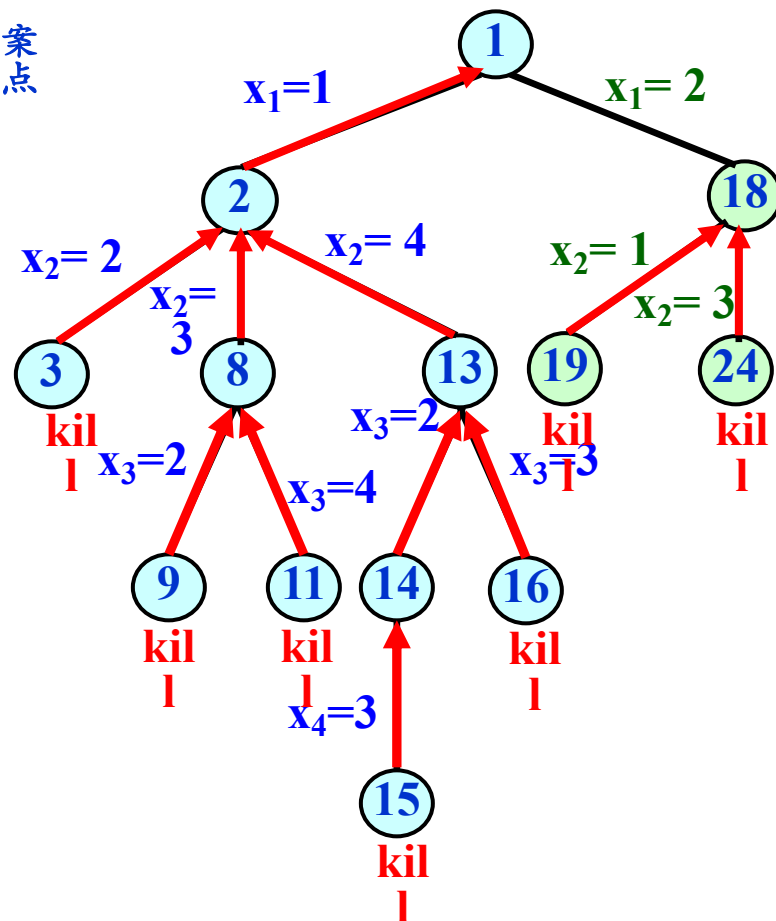
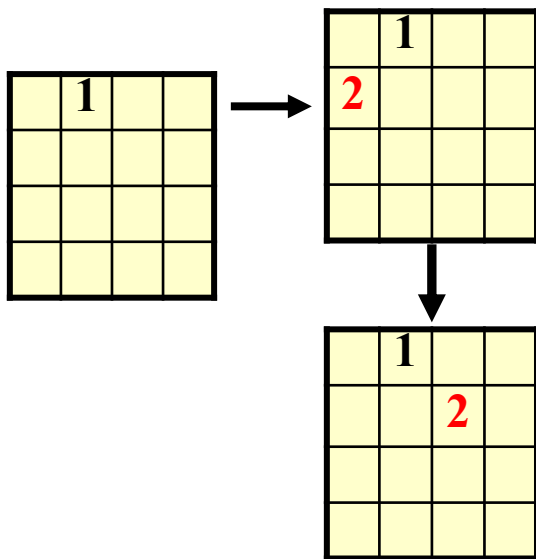
4皇后问题

- 回溯到结点2生成结点13, 路径变为(1, 4), 结点13成为E-结点, 由于它的儿子表示的是一些不可能导致答案结点的棋盘格局, 因此结点13也被杀死, 应回溯。

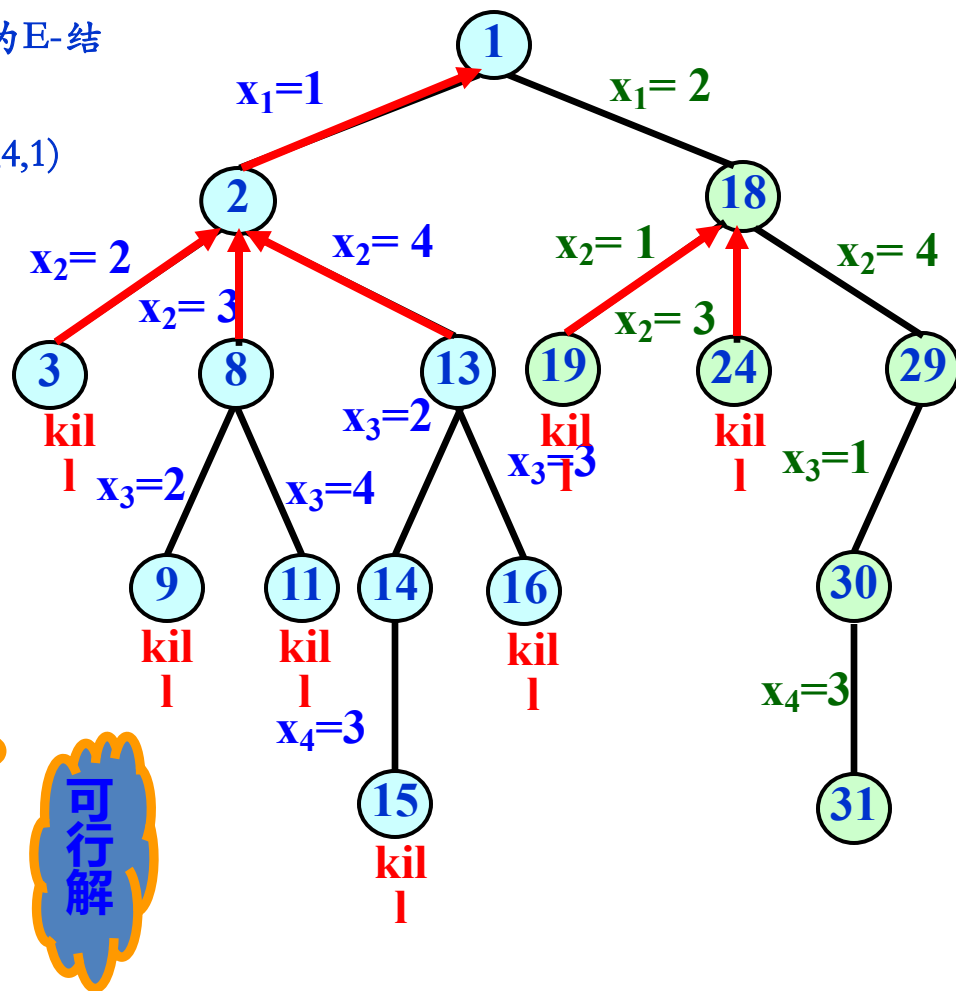
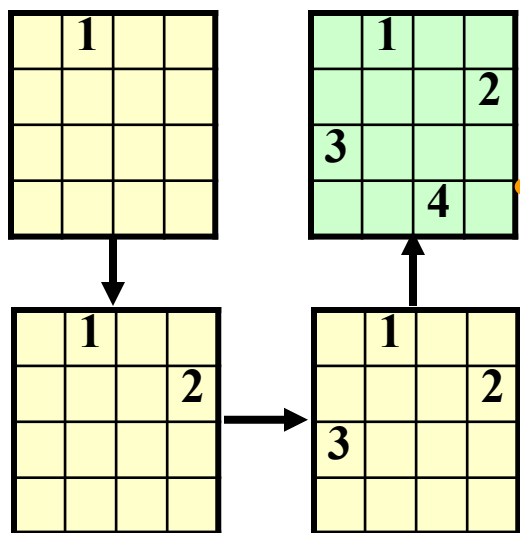


4皇后问题

- 结点2的所有儿子表示的都是不可能导致答案的棋盘格局, 因此结点2也被杀死; 再回溯到结点1生成结点18, 路径变为(2)。
- 结点18的子结点19、结点24被杀死, 应回溯。

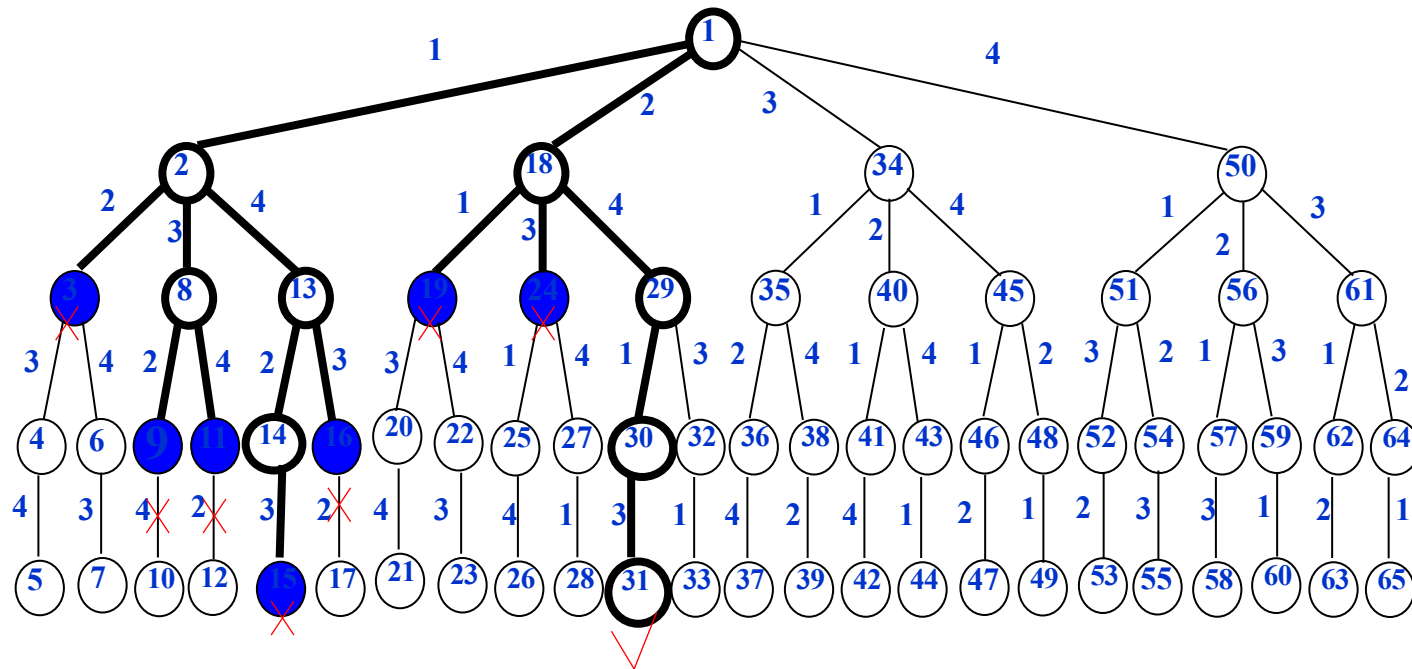


- 结点18生成结点29, 结点29成为E-结点, 路径变为(2,4);
- 结点29生成结点30, 路径变为(2,4,1)
- 结点30生成结点31, 路径变为(2,4,1,3), 找到一个4-王后问题的可行解



4皇后问题

n=4的n皇后问题的搜索、剪枝与回溯



解

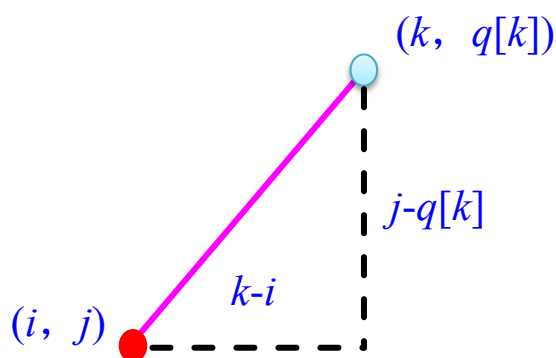
- 解空间是一棵子集树（每个皇后在 $1 \sim n$ 列中找到一个适合的列号，即 n 选一），并且要求所有解。
- 采用整数数组 $q[N]$ 存放 n 皇后问题的求解结果，因为每行只能放一个皇后， $q[i]$ ($1 \leq i \leq n$) 的值表示第 i 个皇后所在的列号

	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

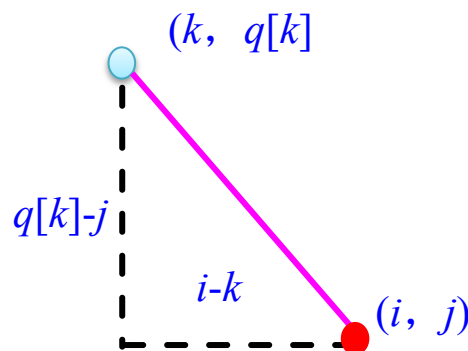
$q[1..6] = \{2, 4, 6, 1, 3, 5\}$

若在 (i, j) 位置上放第 i 个的皇后，是否与已放好的 $i-1$ 个皇后 $(k, q[k])$ 有冲突呢？

- ① 如果 (i, j) 位置与前面某个皇后同列，则有 $q[k]==j$ 成立。
- ② 如果 (i, j) 位置与前面某个皇后同对角线，则恰好构成一个等腰直角三角形，即有 $|q[k]-j|==|i-k|$ 成立。



(a) 对角线1



(b) 对角线2

归纳起来只要 (i, j) 位置满足以下条件，则存在冲突，否则不冲突：

$$(q[k]==j) \parallel (\text{abs}(q[k]-j)==\text{abs}(i-k))$$

$$1 \leq k \leq i-1$$

冲突条件: $(q[k]==j) \parallel (\text{abs}(q[k]-j)==\text{abs}(i-k)) \quad 1 \leq k \leq i-1$

```
bool place(int i,int j)           //测试(i,j)位置能否放置皇后
{   if (i==1) return true;       //第一个皇后总是可以放置
    int k=1;
    while (k<i)                  //k=1~i-1是已放置了皇后的行
    {   if ((q[k]==j) || (abs(q[k]-j)==abs(i-k)))
        return false;
        k++;
    }
    return true;
}
```

求解皇后问题所有解的递归模型

queen(i , n) $\equiv$ n 个皇后放置完毕，输出一个解

若 $i > n$

queen(i , n) $\equiv$ 在第 i 行找到一个合适的位置 (i, j) ,
放置一个皇后;

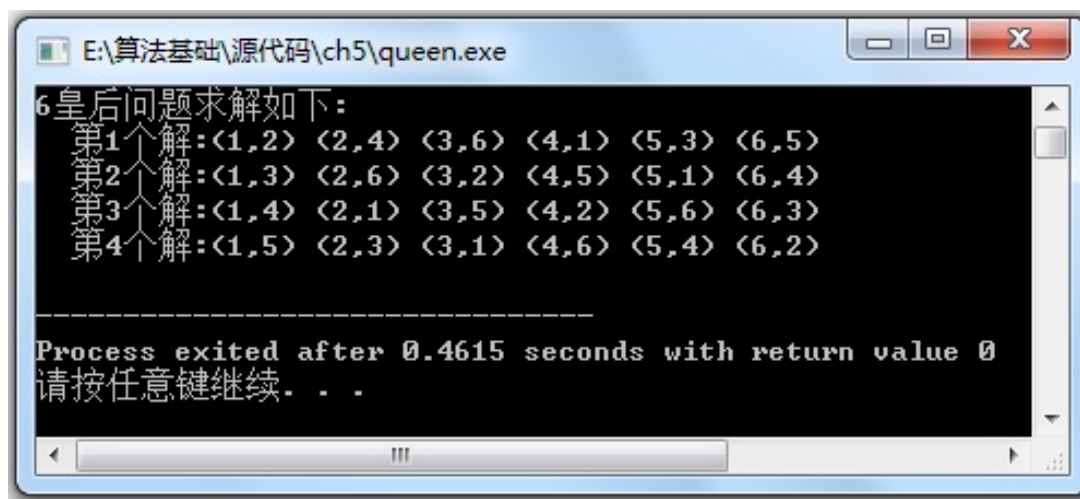
其他

queen($i+1$, n);

<code>int q[MAXN];</code>	//存放各皇后所在的列号,为全局变量
<code>int cnt=0;</code>	//累计解个数
 <code>void queen11(int n,int i)</code>	 //回溯算法
<code>{ if (i>n)</code>	
<code> disp(n);</code>	//所有皇后放置结束
<code> else</code>	
<code> { for (int j=1;j<=n;j++)</code>	//在第i行上试探每一个列j
<code> { if (place(i,j))</code>	//在第i行上找到一个合适位置(i,j)
<code> { q[i]=j;</code>	
<code> queen11(n,i+1);</code>	
<code> q[i]=0;</code>	//回溯
<code> }</code>	
<code> }</code>	
<code> }</code>	
 <code>void queen1(int n)</code>	 //递归法求解n皇后问题
<code>{</code>	
<code> queen11(n,1);</code>	
<code>}</code>	

最坏情况下算法时间复杂度为 $O(n^n)$ 。

程序 验证



The screenshot shows a Windows command prompt window with the title bar "E:\算法基础\源代码\ch5\queen.exe". The window contains the following text:

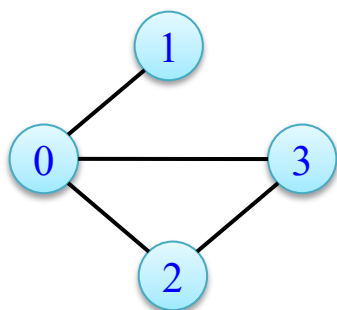
```
6皇后问题求解如下:  
第1个解:<1,2> <2,4> <3,6> <4,1> <5,3> <6,5>  
第2个解:<1,3> <2,6> <3,2> <4,5> <5,1> <6,4>  
第3个解:<1,4> <2,1> <3,5> <4,2> <5,6> <6,3>  
第4个解:<1,5> <2,3> <3,1> <4,6> <5,4> <6,2>  
  
-----  
Process exited after 0.4615 seconds with return value 0  
请按任意键继续. . .
```

5.2.3 图的 m 着色

- 给定无向连通图 G 和 m 种不同的颜色。用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色。如果有一种着色法使 G 中每条边的两个顶点着不同颜色，则称这个图是 m 可着色的。
- 图的 m 着色问题是对于给定图 G 和 m 种颜色，找出所有不同的着色法。

解

- 含 n 个顶点的无向连通图 G ，顶点编号是 $0 \sim n-1$.采用`vector<int>`数组的邻接表 A 存储，其中 $A[i]$ 向量为顶点 i 的所有相邻顶点。
- m 种颜色的编号为 $0 \sim m-1$ 。题目就是为每个顶点 i 选择 m 种颜色中的一种（ m 选一），使得任意两个相邻顶点的着色不同，所以解空间树看成是一棵子集树，并且求解个数，属于求所有解类型。

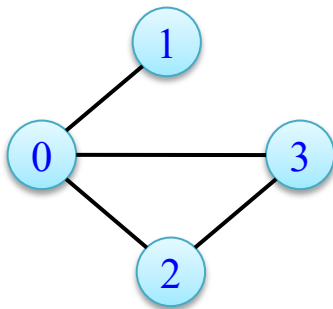


$A[0..3] = \{\{1,2,3\}, \{0\}, \{0,3\}, \{0,2\}\}$

- 设计解向量为 $x = (x_0, x_1, \dots, x_{n-1})$ ，其中 x_i 表示顶点 i 的着色 ($0 \leq x_i \leq m-1$)，初始时置 x 的所有元素为 -1 表示所有顶点均没有着色，用 cnt 累计解个数（初始为 0）。
- **顶点 i :** 所有可能的着色 j 为 0 到 $m-1$ 中的一种，如果顶点 i 的所有相邻顶点的颜色均不等于 j ，说明顶点 i 着色 j 是合适的，只要有一个相邻顶点的颜色等于 j ，则顶点 i 着色 j 是不合适的，需要回溯。

```
int cnt;  
int x[MAXN];  
  
bool judge(int i,int j)  
{   for(int k=0;k<A[i].size();k++)  
    {   if(x[A[i][k]]==j)  
        return false;  
    }  
    return true;  
}
```

//全局变量，累计解个数
//全局变量，x[i]表示顶点i的着色
//判断顶点i是否可以着上颜色j
//存在相同颜色的顶点




```
void dfs(int m,int i)
```

```
{  if(i>=n)
```

```
    cnt++;
```

```
    else
```

```
    {  for(int j=0;j<m;j++)
```

```
        {  x[i]=j;
```

```
            if(judge(i,j))
```

```
                dfs(m,i+1);
```

```
            x[i]=-1;
```

```
        }
```

```
    }
```

```
}
```

//递归回溯算法

//达到一个叶子结点

//置顶点i为颜色j

//若顶点i可以着上颜色j

//回溯

<code>int Colors(int m)</code>	<code>//求图的m着色问题</code>
<code>{ cnt=0;</code>	
<code>memset(x,0xff,sizeof(x));</code>	<code>//所以元素初始化为-1</code>
<code>dfs(m,0);</code>	<code>//从顶点0开始搜索</code>
<code>return cnt;</code>	
<code>}</code>	

【算法分析】该算法中每个顶点试探编号为 $0 \sim m-1$ 着色，共 n 个顶点，对应解空间树是一棵 m 叉树（子集树），所有算法的最坏时间复杂度为 $O(m^n)$ 。

5.3 基于排列树框架的问题求解

5.3.1 批处理作业调度问题

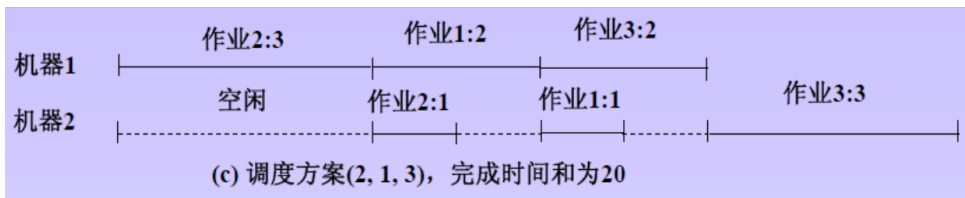
问题描述：给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。每个作业必须先由机器1处理，然后由机器2处理。作业 J 需要机器 j 的处理时间为 t 。对于一个确定的作业调度，设 F_i 是作业 i 在机器 j 上完成处理的时间。所有作业在机器2上完成处理的时间和称为该作业调度的完成时间和。

批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案，使其完成时间和达到最小。

t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

- 这个3个作业的6种可能的调度方案是：
(1,2,3) (1,3,2) (2,1,3)
(2,3,1) (3,1,2) (3,2,1)
- 对应的完成时间和分别是：
19,18,20,21,19,19。
- 最佳调度方案是1,3,2，其完成时间和为18。

批处理作业调度问题



t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

➤ 这个3个作业的6种可能的调度方案是：

(1,2,3) (1,3,2) (2,1,3)

(2,3,1) (3,1,2) (3,2,1)

➤ 对应的完成时间和分别是：

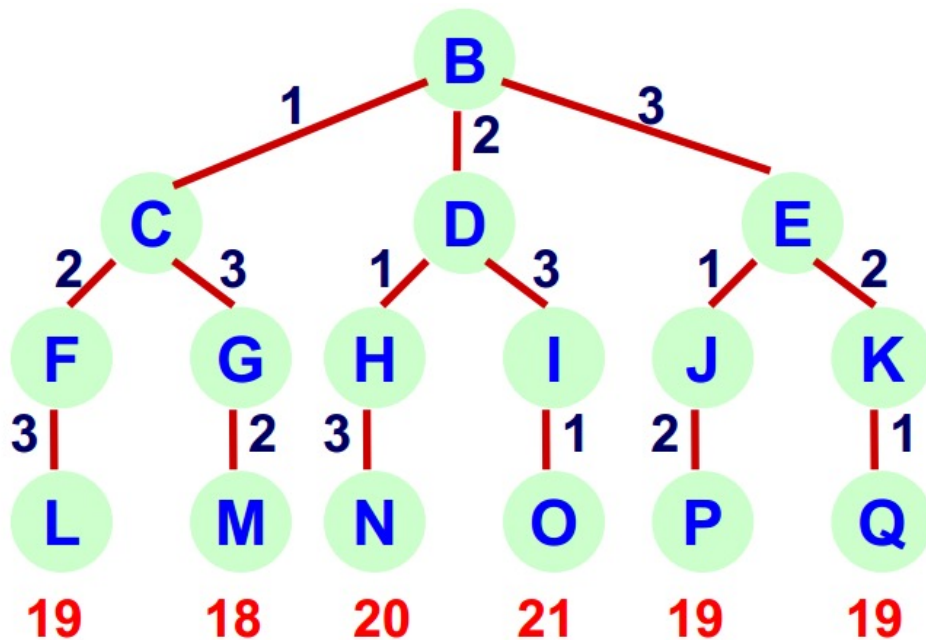
19,18,20,21,19,19。

➤ 最佳调度方案是1,3,2，其完成时间和为18。

批处理作业调度问题

显然，批处理作业的一个最优调度应使机器1没有空闲时间，且机器2的空闲时间最小。

可以证明，
存在一个最优作业调度使得在机器1和机器2上作业以相同次序完成。



批处理作业调度问题要从 n 个作业的所有排列中找出有最小完成时间和的作业调度，所以批处理作业调度问题的解空间是一颗排列树。

子树随着作业的减少而减少。

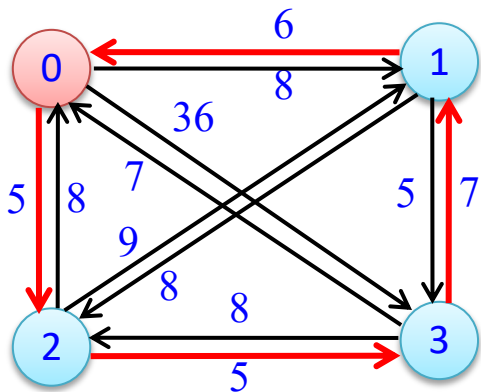
```
class Flowshop {  
    friend Flow(int**, int, int []);  
    private:  
        void Backtrack(int i);  
        int **M, // 各作业所需的处理时间  
        *x, // 当前作业调度  
        *bestx, // 当前最优作业调度  
        *f2, // 机器2完成处理时间  
        f1, // 机器1完成处理时间  
        f, // 完成时间和  
        bestf, // 当前最优值  
        n; // 作业数};
```

批处理作业调度问题

```
void Flowshop::Backtrack(int i)
{
    if (i > n) {
        for (int j = 1; j <= n; j++)
            bestx[j] = x[j];
        bestf = f;
    }
    else
        for (int j = i; j <= n; j++) {
            f1+=M[x[j]][1];
            f2[i]=((f2[i-1]>f1)?f2[i-1]:f1)+M[x[j]][2];
            f+=f2[i];
            if (f < bestf) {
                Swap(x[i], x[j]);
                Backtrack(i+1);
                Swap(x[i], x[j]);
            }
            f1-=M[x[j]][1];
            f-=f2[i];
        }
}
```

5.3.2 货郎担问题 (TSP)

假设有一个货郎担要拜访 n 个城市，他必须选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市，要求**路径长度最短的路径**。



路径1: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$: 28

路径2: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$: 29

路径3: $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 0$: 26

路径4: $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$: 23

路径5: $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$: 59

路径6: $0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 0$: 59

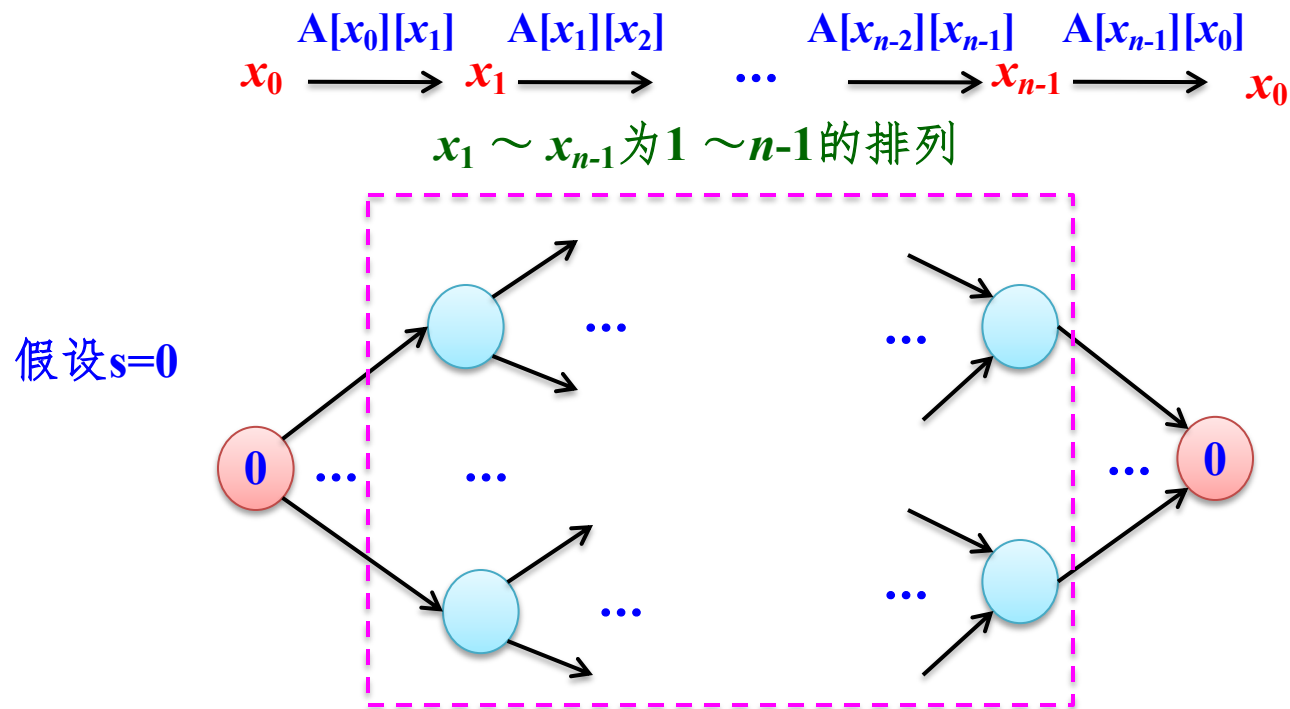
旅行计划 (LintCode1891★★)

解

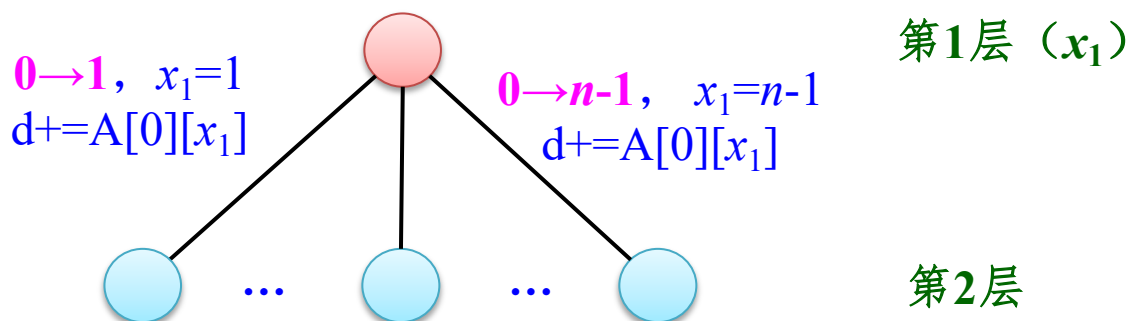
- 用邻接矩阵 A 存储带权图。
- 起点为 s 。
- 设计解向量 $x = (x_0, x_1, \dots, x_{n-1})$ ，每个 x_i 表示一个图中顶点，实际上每个 x 表示一条路径，初始时 x_0 置为起点 s ， $x_1 \sim x_{n-1}$ 为其他 $n-1$ 个顶点编号。

算法 $\text{dfs}(x, d, s, i)$ 基于排列树框架，设计的几个重点如下：

- d 表示当前路径的长度，用 bestx 保存最短路径， bestd 表示最短路径长度，其初始值置为 ∞ 。
- x_0 固定作为起点 s ，不能取其他值，所以应该从 $i=1$ （此时 $d=0$ ）开始调用 dfs 。



- 假设 $s=0$, x 初始时为 $(0, 1, \dots, n-1)$, $i=1$ 时 x_1 会取 $x[1..n-1]$ 的每一个值: 当 $x_1=x_1$ (1) 时, 对应路径长度为 $d+A[0][1]$, 当 $x_1=x_2$ (2) 时, 对应路径长度为 $d+A[0][2]$, 以此类推。



- 第 i 层: x_i 取 $x[i..n-1]$ 中的某个值后当前路径长度为 $d+A[x[i-1]][x[i]]$ 。

- 当搜索到达某个叶子结点时 ($i \geq n$)，对应的TSP路径长度应该是 $d + A[x[n-1]][s]$ （因为TSP路径是闭合的回路），对应的路径是 $x \cup \{s\}$ 。通过长度的比较求最优解。
- **剪支：**若当前已经求出最短路径长度 $bestd$ ，如果 x_i 取 x_j 值，对应的路径长度为 $d + A[x[i-1]][x[j]]$ ，仅仅扩展满足 $d + A[x[i-1]][x[j]] < bestd$ 的路径。

```

vector<vector<int>> A={{0,8,5,36},{6,0,8,5},{8,9,0,5},{7,7,8,0}};
int n=4;
int cnt=0;                                     //路径条数累计
vector<int> bestx;                             //保存最短路径
int bestd=INF;                                //保存最短路径长度，初始为 $\infty$ 

void disp(vector<int>&x,int d,int s)           //输出一个解
{
    printf(" 第%d条路径: ",++cnt);
    for (int j=0;j<x.size();j++)
        printf("%d->",x[j]);
    printf("%d",s);                            //末尾加上起点s
    printf(", 路径长度: %d\n",d+A[x[n-1]][s]);
}

```

```
void dfs(vector<int>&x,int d,int s,int i)
```

```
{  if(i>=n)
```

```
    {  disp(x,d,s);
```

```
        if(d+A[x[n-1]][s]<bestd)
```

```
        {  bestd=d+A[x[n-1]][s];
```

```
            bestx=x;
```

```
            bestx.push_back(s);
```

```
        }
```

```
    }
```

```
    else
```

```
    {  for(int j=i;j<n;j++)
```

```
        {  if (A[x[i-1]][x[j]]!=0 && A[x[i-1]][x[j]]!=INF)
```

```
            {  if(d+A[x[i-1]][x[j]]<bestd)
```

```
                {  swap(x[i],x[j]);
```

```
                    dfs(x,d+A[x[i-1]][x[i]],s,i+1);
```

```
                    swap(x[i],x[j]);
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

//回溯法算法

//到达一个叶子结点

//输出一个解

//同时比较求最优解

//求TSP长度

//更新bestx

//末尾添加起始点

//没有到达叶子结点

//试探x[i]走到x[j]的分支

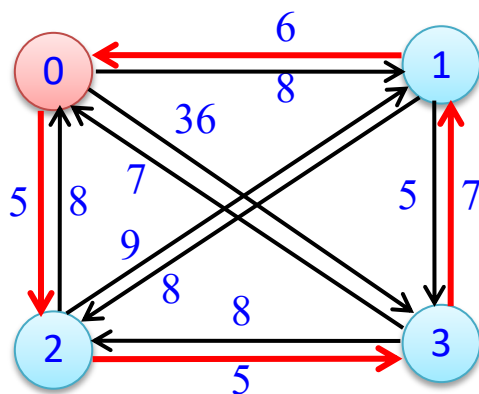
//若x[i-1]到x[j]有边

//剪支

void TSP1(int s)	//回溯法求解TSP(起始点为s)
{ vector<int> x;	//定义解向量
x.push_back(s);	
for(int i=1;i<n;i++)	//将非s的顶点添加到x中
if(i!=s) x.push_back(i);	
int d=0;	
dfs(x,d,s,1);	//从x[1]顶点开始扩展
}	

【算法分析】 算法的解空间是一棵排列树，由于是从第一层开始搜索的，树的高度为 n ，所以最坏的时间复杂度为 $O((n-1)!)$ 。

程序验证



路径1: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$: 28

路径2: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$: 29

路径3: $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 0$: 26

路径4: $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$: 23

路径5: $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$: 59

路径6: $0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 0$: 59

```
E:\算法基础\源代码\ch5\TSP.exe
求解结果
第1条路径: 0->1->2->3->0, 路径长度: 28
第2条路径: 0->1->3->2->0, 路径长度: 29
第3条路径: 0->2->1->3->0, 路径长度: 26
第4条路径: 0->2->3->1->0, 路径长度: 23
最短路径: 0->2->3->1->0
路径长度: 23

-----
Process exited after 0.4435 seconds with return value 0
请按任意键继续. . .
```


THANKS!



温故而知新